

Quantum assistant: Specialization of multimodal vision-language models for quantum computing

Samuel Lima Braz , João Paulo Reus Rodrigues Leite *

Institute of Systems Engineering and Information Technologies, Federal University of Itajubá (UNIFEI), Av. BPS, 1303 – Pinheirinho, Itajubá, MG, 37500-903, Brazil

ARTICLE INFO

Keywords:

Multimodal dataset
Synthetic data generation
Vision-language models
Parameter-efficient fine-tuning
Quantum computing
Qiskit
Code generation
Low-rank adaptation

ABSTRACT

Code assistants for quantum computing operate exclusively on text, disregarding visual representations (circuit diagrams, Bloch spheres, and histograms) that permeate the domain's documentation. This work is presented as a resource-and-pilot study with three contributions: (1) the first public multimodal dataset for Qiskit-oriented quantum programming, containing 8366 samples (45% with images) across function completion, code generation, and question answering tasks, released under Apache 2.0; (2) a reproducible synthetic generation pipeline that extracts content from public Qiskit documentation, transcribes images via a multimodal model, generates question-answer pairs, and validates code through automatically generated unit tests; and (3) a feasibility study of parameter-efficient VLM specialization, in which rsLoRA (rank 32, two epochs) on Qwen3-VL-8B-Instruct is retained as the best configuration under a controlled method-selection comparison of five LoRA variants. On the text-only external benchmarks Qiskit HumanEval and Qiskit HumanEval Hard, which measure quantum-code specialization rather than visual understanding, the fine-tuned model reaches 41.72% and 30.46% Pass@1 respectively, above the IBM-specialized Qwen2.5-Coder-14B-Qiskit (40.40% / 25.83%) and granite-3.3-8b-qiskit (17.88% / 9.93%), and +17.22 / +15.89 pp over the Qwen3-VL-8B-Instruct base. Visual capability is assessed on the synthetic multimodal test split, where multimodal samples reach 64.29% Pass@1 versus 46.33% on text-only samples (+17.96 pp); a matched training ablation that withholds the multimodal portion of the training set localizes +6.25 pp of that gap to the multimodal training signal. The dataset, models, and code are available under the Apache 2.0 license.

1. Introduction

Quantum computing exploits phenomena from quantum mechanics — superposition, entanglement, and interference — to process information in ways fundamentally distinct from classical computers. This technology promises to impact strategic sectors: molecular simulation for drug discovery (Cerezo et al., 2021), financial optimization (Herman et al., 2023), and post-quantum cryptography (Nielsen & Chuang, 2010). However, the transition from theory to practice encounters considerable technical complexity.

Programming quantum algorithms requires knowledge that rarely coexists in a single professional: quantum mechanics fundamentals to understand the behavior of qubits and gates, linear algebra to manipulate states and operators, and mastery of specialized frameworks for effective implementation. Qiskit (Qiskit contributors, 2024a), developed by IBM, stands out as the leading open-source SDK for quantum computing, offering tools for circuit construction, transpilation to specific hardware, and execution on real quantum processors. This intersection

of skills creates a significant barrier to entry that limits the field's expansion (Murillo et al., 2024).

Code assistants based on Generative Artificial Intelligence offer a viable path to democratize access to quantum programming. Large Language Models (LLMs) trained on vast code repositories have demonstrated the ability to assist developers through intelligent autocompletion, function generation from textual descriptions, and explanation of existing implementations (Chen et al., 2021). Recognizing this potential, IBM developed the Qiskit Code Assistant (Dupuis et al., 2024), specializing models from the Granite family for quantum code generation, achieving 46.5% Pass@1 on the Qiskit HumanEval benchmark and demonstrating the viability of domain specialization for quantum code assistants.

Despite this advance, a limitation persists: existing solutions process exclusively text, not exploiting the fundamental visual representations of the domain. Quantum circuits are represented as diagrams of lines and gates; qubit states are visualized on Bloch spheres; measurement results appear as probability histograms. These visual elements permeate

* Corresponding author.

E-mail addresses: samuel.limabraz@unifei.edu.br (S.L. Braz), joapaulo@unifei.edu.br (J.P.R.R. Leite).

the official documentation, scientific papers, and educational materials of the field. The Qiskit documentation itself relies on visualizations to explain concepts and demonstrate results (Qiskit contributors, 2024a). An effective assistant should comprehend both code and its graphical representations.

Vision-Language Models (VLMs) offer precisely this visual-textual fusion capability. Recent architectures such as Qwen3-VL (Bai et al., 2025), InternVL 3.5 (Chen et al., 2025), and Gemma 3 (Gemma Team, 2025) demonstrate competitive performance on tasks requiring visual-linguistic comprehension, including interpretation of technical diagrams and reasoning over complex images. However, there exists, to date, no public dataset or specialized model integrating vision and language for Qiskit-oriented quantum programming — a gap that this work seeks to fill.

The absence of visual capability in current quantum code assistants follows from two complementary limitations of existing components. Existing assistants such as the Qiskit Code Assistant (Dupuis et al., 2024) and the post-trained Llama-based models of Kain et al. (2025) are built on text-only decoder LLMs (Granite, Llama-3.1), which have no visual encoder and therefore no architectural path through which circuit diagrams, Bloch spheres, or measurement histograms can enter the model. Off-the-shelf VLMs solve the architectural problem but present a complementary limitation: their visual pre-training emphasizes natural images, documents, and general charts (e.g. the ChartQA benchmark Masry et al., 2022), and does not cover the conventions that are specific to quantum documentation — gate boxes and single- or double-line wire notation for qubit and classical registers in circuit diagrams, axis and state-vector conventions in Bloch spheres, or bitstring-indexed bars in measurement histograms. Bridging these two limitations requires paired (*quantum visualization, code, concept*) data, which is the specific training signal that no public dataset has provided; this is the gap the present work targets.

1.1. Motivation

Three factors motivate this work:

Scarcity of visual data: The Qiskit Code Assistant demonstrated the effectiveness of specialized LLMs, yet its training dataset remains proprietary and exclusively textual. There is no public availability of data combining quantum code with corresponding visualizations, impeding research into visual-linguistic approaches.

Limitation of text-only solutions: Common usage scenarios include interpreting a circuit diagram from a scientific paper and implementing it in code, or debugging an algorithm by comparing its visualization with the original specification. Assistants that process only text do not effectively aid in these tasks, which constitute a substantial part of the workflow in quantum research and development.

Feasibility of specialization: Full fine-tuning of multimodal models with billions of parameters demands computational resources prohibitive for academic research groups. Parameter-Efficient Fine-Tuning (PEFT) techniques, such as Low-Rank Adaptation (LoRA) (Hu et al., 2022) and its variants, enable specialization with accessible hardware, but their application to VLMs for technical domains remains underexplored.

1.2. Objectives and contributions

This work presents three contributions, with the dataset and pipeline as the primary artifacts:

1. **Public multimodal dataset:** The first public multimodal dataset for Qiskit-oriented quantum programming, with 8366 samples distributed across three formats — function completion, code generation, and question answering — with balance between textual and visual inputs, made available under the Apache 2.0 license. Code tasks target Qiskit ≥ 2.0 ; conceptual question answering also covers

general quantum computing notions present in the source corpus, without claiming coverage of other quantum SDKs.

2. **Synthetic generation pipeline:** A reproducible pipeline for constructing the above dataset, with executable validation and multi-stage curation. The process automates knowledge extraction from public Qiskit documentation and related educational materials, integrating visual transcription via VLM and code verification through automatically generated unit tests, documented in sufficient detail for reuse in other specialized technical domains.
3. **Feasibility study of VLM specialization:** A controlled evaluation of PEFT techniques applied to an open-source VLM under a method-selection protocol, demonstrating that the dataset enables effective domain specialization and that the resulting model adds a capability — vision-grounded Qiskit code understanding — that is absent in existing text-only quantum code assistants.

The fine-tuning experiments show that the dataset supports effective domain specialization; they are not intended as a benchmark of VLM architectures or as a full study of PEFT variants. The single base model, the controlled PEFT comparison, and the restricted hyperparameter search are stated explicitly in Section 3.4.

2. Literature review

This section presents the theoretical foundations necessary for understanding this work, organized into five main axes: Transformer architecture and language models, VLMs, parameter-efficient fine-tuning techniques, quantum computing and the Qiskit framework, and related work in the area.

2.1. Transformer architecture and language models

The Transformer architecture, introduced by Vaswani et al. (2017), established a fundamental paradigm in natural language processing by eliminating recursive dependencies and replacing them with purely position-based attention mechanisms. This architectural innovation enabled efficient parallel processing and scalability to large-scale models.

The central component is the scaled dot-product attention mechanism. For an input sequence $X \in \mathbb{R}^{n \times d}$, where n denotes the sequence length and d the model dimension, attention is computed through three linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learned weight matrices. The attention operation is then defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

The scaling factor $\sqrt{d_k}$ prevents large dot products from saturating the softmax function, stabilizing gradients during training.

Multi-head attention allows the model to capture different types of dependencies simultaneously through multiple parallel projections:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (3)$$

where each head $\text{head}_i = \text{Attention}(XW_{Q_i}^i, XW_{K_i}^i, XW_{V_i}^i)$ operates in subspaces of dimension $d_k = d/h$, and $W_O \in \mathbb{R}^{hd_k \times d}$ projects the concatenation back to the original space.

Large-scale language models such as GPT (Brown et al., 2020) and LLaMA (Touvron et al., 2023) employ the decoder-only variant of the Transformer, characterized by the use of a causal mask that ensures prediction of the token at position i depends exclusively on tokens at positions $j < i$. Training occurs in multiple phases: autoregressive pre-training on a massive corpus, followed by supervised fine-tuning and alignment with human preferences through Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022).

2.2. Vision-language models

Vision-Language Models (VLMs) extend the capability of LLMs to process and comprehend visual and textual information, integrating representations from both modalities into a unified space. The typical architecture comprises three main components: a visual encoder, a projection module, and a base language model (Liu et al., 2023).

The visual encoder transforms images into sequences of vector representations. Modern approaches are based on Vision Transformers (ViT) (Dosovitskiy et al., 2021), which divide images into non-overlapping patches and process them as sequences. Given an image $I \in \mathbb{R}^{H \times W \times C}$ divided into $N = (H \times W)/P^2$ patches of size $P \times P$, the encoder produces a sequence of embeddings $Z_v \in \mathbb{R}^{N \times d_v}$, where d_v is the representation dimension. Parallel designs that combine convolution with self-attention to strengthen local and mid-level visual structure, such as Dynamic Unary Convolution in Transformers (Duan et al., 2023), illustrate ongoing work on efficient adaptive visual pathways inside Transformer-style encoders; the present study uses an off-the-shelf VLM encoder and does not modify that architecture.

Alternatives include architectures such as SigLIP (Zhai et al., 2023), which uses a sigmoid loss function for contrastive pre-training, improving training efficiency and representation quality.

The projection module performs alignment between the visual representation space and the language model's embedding space. Strategies range from simple linear transformations to more sophisticated architectures such as the Q-Former from BLIP-2 (Li et al., 2023), which employs cross-attention layers to filter and compress relevant visual information before projection.

Recent architectures demonstrate significant advances in visual-linguistic fusion capabilities. Qwen3-VL (Bai et al., 2025) introduces dynamic resolution mechanisms and enhanced visual reasoning, enabling efficient processing of high-resolution images. InternVL 3.5 (Chen et al., 2025) adopts a joint pre-training approach, unifying language and vision learning from the initial stages, resulting in better integration between modalities. Gemma 3 (Gemma Team, 2025) presents an optimized hybrid architecture with an extended context window, facilitating comprehension of extensive technical documents.

2.3. Parameter-efficient fine-tuning

Full fine-tuning of modern LLMs with billions of parameters demands computational and memory resources prohibitive for most research and application scenarios. Parameter-Efficient Fine-Tuning (PEFT) techniques emerged as a viable alternative, enabling model adaptation to specific tasks by modifying only a minimal fraction of parameters while maintaining competitive performance (Ding et al., 2023).

2.3.1. Low-rank adaptation (LoRA)

Low-Rank Adaptation (LoRA) (Hu et al., 2022) is grounded in the hypothesis that weight updates during fine-tuning reside in low-dimensional subspaces. This observation is theoretically supported by the concept of intrinsic dimensionality (Aghajanyan et al., 2021), which demonstrates that large models can be effectively adapted in spaces of dimension significantly smaller than the full parameter space.

For a weight matrix $W_0 \in \mathbb{R}^{d \times k}$ of a layer in the base model, LoRA decomposes the update ΔW as:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \quad (4)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices with $r \ll \min(d, k)$, and α is a scaling hyperparameter. During training, only A and B are updated, while W_0 remains frozen. The number of trainable parameters reduces from dk to $r(d + k)$, typically representing less than 1% of total parameters.

2.3.2. LoRA variants

Several variants have been proposed to enhance specific aspects of the original LoRA:

Rank-stabilized LoRA (rsLoRA) (Kalajdziewski, 2023) modifies the scaling factor from α/r to $\alpha/\sqrt{r}$, stabilizing gradients and enabling efficient use of higher ranks without performance degradation. This modification is particularly beneficial in configurations with $r > 64$.

Weight-decomposed low-rank adaptation (DoRA) (Liu et al., 2024) decomposes weights into magnitude and direction components. The magnitude is learned directly, while the direction is parameterized via LoRA:

$$W = m \frac{W_0 + BA}{\|W_0 + BA\|_c} \quad (5)$$

where m is a learnable scalar and $\|\cdot\|_c$ denotes the column norm. This decomposition enables independent control over magnitude and direction, resulting in better performance especially at low ranks.

Principal singular values and singular vectors adaptation (PiSSA) (Meng et al., 2024) initializes adapters using singular value decomposition (SVD) of the original weights. The principal singular values and corresponding singular vectors are used to initialize A and B , accelerating convergence and improving final performance.

Orthonormal low-rank adaptation (OLoRA) (Buyukakyuz, 2024) applies orthogonal QR decomposition to better preserve the structure of original weights during adaptation. This approach demonstrates greater training stability and faster convergence across various benchmarks.

2.3.3. Application of PEFT to VLMs

Applying PEFT techniques to VLMs requires specific considerations about which components to adapt. Common strategies include: (1) adapting only the visual-linguistic projection module, keeping the visual encoder and LLM frozen; (2) applying LoRA adapters to the attention and feed-forward layers of the LLM; (3) hybrid combinations that adapt multiple components simultaneously (Gao et al., 2023). The choice of strategy depends on the trade-off between computational efficiency, memory usage, and performance on the target task. Recent work has also explored PEFT for multimodal generative vision models through non-linear parameter transforms inspired by Möbius geometry (Duan et al., 2025); that line of work targets generative vision adaptation (for example, controllable image generation) rather than instruction-tuned quantum code assistants, but it reinforces that parameter-efficient adaptation of multimodal models remains an active research area.

2.4. Quantum computing and the qiskit framework

Quantum computing exploits phenomena from quantum mechanics — superposition, entanglement, and interference — to process information in a fundamentally distinct manner from classical computers. The fundamental element is the qubit, a two-level quantum system described by a vector in the two-dimensional Hilbert space $\mathcal{H} = \mathbb{C}^2$.

The state of a qubit is represented as a linear superposition:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (6)$$

where $|0\rangle$ and $|1\rangle$ form the orthonormal computational basis and $\alpha, \beta \in \mathbb{C}$ are complex probability amplitudes. Measurement of a qubit collapses the state to $|0\rangle$ with probability $|\alpha|^2$ or $|1\rangle$ with probability $|\beta|^2$.

Quantum operations are represented by unitary matrices that preserve the norm of the state vector. Fundamental gates include Pauli rotations (X, Y, Z), the Hadamard gate (H) which creates uniform superposition, and two-qubit gates such as CNOT which establish entanglement. Quantum circuits compose sequences of these operations to implement algorithms, with visual representation facilitating comprehension and debugging.

Qiskit (Qiskit contributors, 2024a) is the leading open-source SDK for quantum computing, offering a modular architecture that covers the complete development cycle:

- `qiskit.circuit`: Construction and manipulation of quantum circuits, including classes for gates, parameters, and visualization.

- `qiskit.primitives`: Standardized interfaces for expectation value estimation (`EstimatorV2`) and sampling of measurement distributions (`SamplerV2`).
- `qiskit.transpiler`: Circuit optimization and compilation for specific hardware, including qubit mapping, gate optimization, and noise mitigation.
- `qiskit_ibm_runtime`: Execution on real quantum processors through the IBM Quantum platform, with support for sessions, error mitigation, and optimized primitives.

Starting from version 2.0, Qiskit introduced significant architectural changes, including deprecation of functions such as `transpile()` and `execute()` in favor of primitives and pass managers, requiring legacy code updates and specialized knowledge for effective migration.

2.5. Related work

2.5.1. Evaluation benchmarks

The Qiskit HumanEval benchmark (Vishwakarma et al., 2024), inspired by the original HumanEval for Python (Chen et al., 2021), establishes a standard for evaluating quantum code generation models. The dataset comprises 151 function completion problems, each accompanied by unit tests for automatic correctness verification. The Pass@ k metric quantifies the probability that at least one among k generated solutions passes all tests:

$$\text{Pass@}k = \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (7)$$

where n is the total number of solutions generated per problem and c the number of correct solutions. The benchmark stratifies problems into three difficulty levels: basic (54 problems), intermediate (45), and advanced (2), enabling granular performance analysis.

An extension, Qiskit HumanEval Hard, presents code generation tasks from natural language descriptions, evaluating more general capabilities for comprehension and implementation of quantum algorithms.

2.5.2. Qiskit code assistant

The Qiskit Code Assistant (Dupuis et al., 2024) represents the seminal work in specializing LLMs for quantum code. The project trained models from the Granite family (based on the GPT-BigCode architecture) using datasets extracted from official tutorials, documentation, and Qiskit GitHub repositories. The collection process involved automatic crawling for repositories containing “qiskit” in name or description, filtering of deprecated code (keeping only samples updated after 2022), and exact deduplication.

The Granite-8B-Qiskit model, with 8 billion parameters, achieved 46.5% Pass@1 on Qiskit HumanEval (Dupuis et al., 2024), surpassing larger generalist models such as GPT-3.5. The work demonstrated that domain specialization through curated data is more effective than simply increasing model size. However, the training dataset remains proprietary, limiting reproducibility and extension by other researchers.¹

2.5.3. Continued post-training

Kain et al. (2025) investigated the impact of continued post-training on quantum code, using Llama-3.1 models as the base. The work demonstrated that models specialized through post-training on a quantum corpus outperform GPT-4o on Qiskit code generation tasks, evidencing that data quality and relevance are more determinant than absolute volume. The study also explored quantum verifiable rewards techniques for alignment.

¹ The Qiskit/granite-3.3-8b-qiskit checkpoint evaluated in Section 4 is a later 8 B release built on Granite 3.3 and is distinct from the original Granite-8B-Qiskit model whose published Pass@1 is quoted here.

2.5.4. Differentiation of the proposed work

Beyond quantum-code assistants, recent multimodal studies emphasize that how textual and visual inputs are formulated affects downstream multimodal performance. Multi-text guidance with large generative vision-language models has been shown to improve multi-modality image fusion by structuring cross-modal interaction through generated textual descriptions (Wang et al., 2025), and related work on multi-focus image fusion approaches the problem as input-space construction rather than only loss or architecture redesign (Wang et al., 2026). These results are not quantum-programming contributions, but they illustrate that image-bearing inputs and textual or visual guidance are first-class design choices in multimodal systems. The present work applies image-bearing inputs to Qiskit-oriented code and conceptual tasks rather than to image-fusion benchmarks.

The present work differs from previous quantum-code assistants in three fundamental aspects: (1) **Visual-Textual Integration**: while previous works process exclusively text, this work proposes incorporation of visual information (circuit diagrams, Bloch spheres, histograms) essential for quantum computing comprehension; (2) **Data Openness**: the proposed dataset is published under the Apache 2.0 license, facilitating reproducibility and extension; (3) **VLM Specialization**: exploration of PEFT techniques applied to VLMs, an area still underexplored in the context of specialized technical domains.

3. Methodology

The methodology comprises three main components: (1) construction of a synthetic multimodal dataset through an automated extraction and generation pipeline; (2) specialization of VLMs via Parameter-Efficient Fine-Tuning techniques; and (3) quantitative evaluation on established benchmarks and on the proposed dataset.

3.1. Synthetic dataset construction

The dataset is generated from public technical documentation, including official Qiskit documentation, IBM Quantum Learning tutorials, and community educational notebooks. Fig. 1 presents the complete processing pipeline, structured in nine sequential stages with validation and curation mechanisms at multiple points. Fig. 2 details the internal sample generation process, including diversity-aware allocation, input planning, executable validation, and post-processing.

3.1.1. Data sources and licensing

The corpus was constructed exclusively from materials with permissive licenses, organized into three categories. Table 1 summarizes the main sources.

Official documentation and repositories: The main base comes from the Qiskit/documentation repository (Qiskit contributors, 2024b), containing guides, tutorials, and APIs in MDX and Jupyter Notebook format. This base is complemented by Qiskit community repositories, including materials from the Quantum Global Summer School (QGSS 2025), Qiskit Developer Challenge (QDC 2025), and quantum machine learning tutorials. All under Apache 2.0 license.

Scientific papers and books: Ten papers from arXiv and ACM on quantum computing fundamentals, algorithms, and the Qiskit framework were processed, all with CC BY 4.0 license. Books such as *Quantum Computing for the Quantum Curious* (Springer, CC BY 4.0) complement conceptual coverage.

Selection criteria: Content compatible with Qiskit ≥ 2.0 was prioritized, excluding material with deprecated APIs. Collection occurred in November 2025. The combination of Apache 2.0 and CC BY 4.0 licenses allows redistribution of the dataset under Apache 2.0.

3.1.2. Document ingestion and analysis

The initial stage processes documents in three distinct formats, each with a specific extraction strategy:

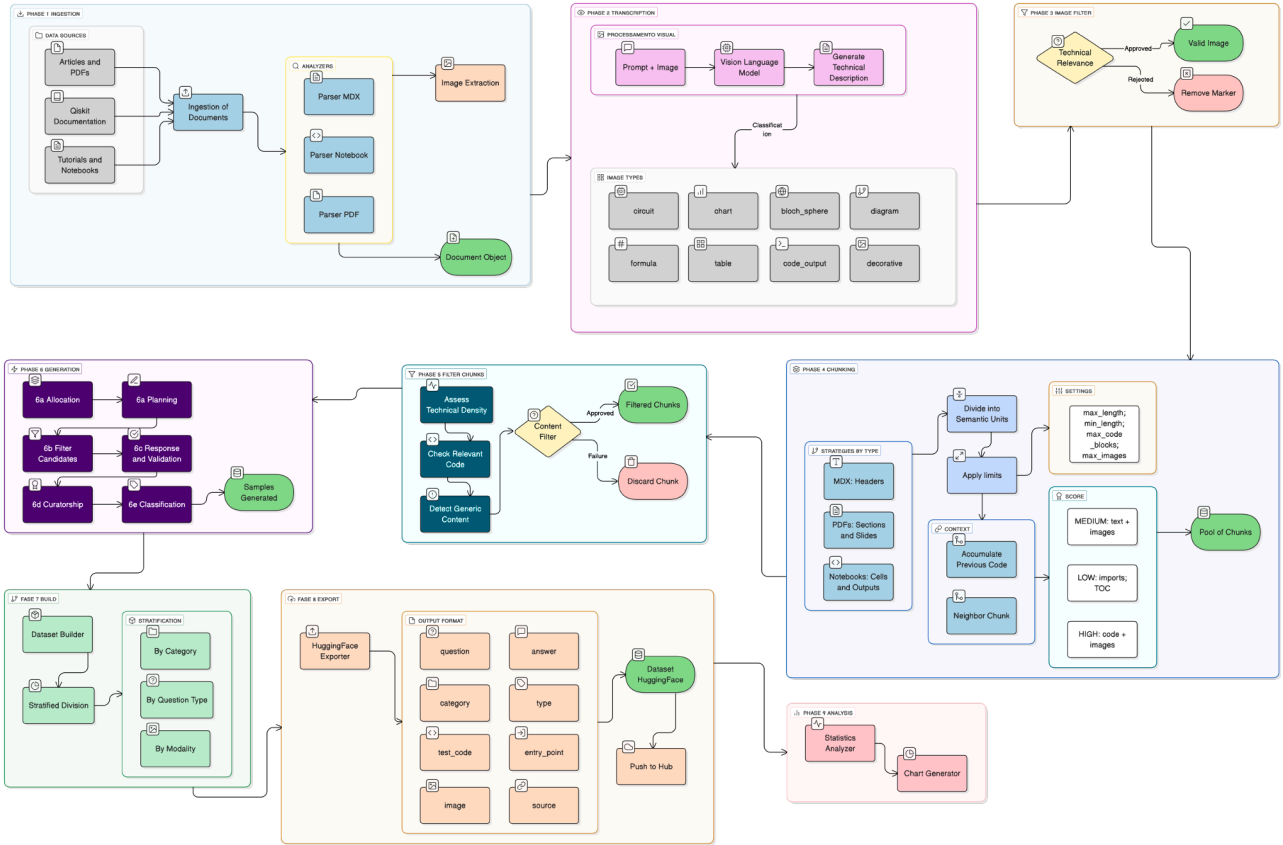


Fig. 1. Complete pipeline for synthetic multimodal data generation, from document ingestion to final dataset export.

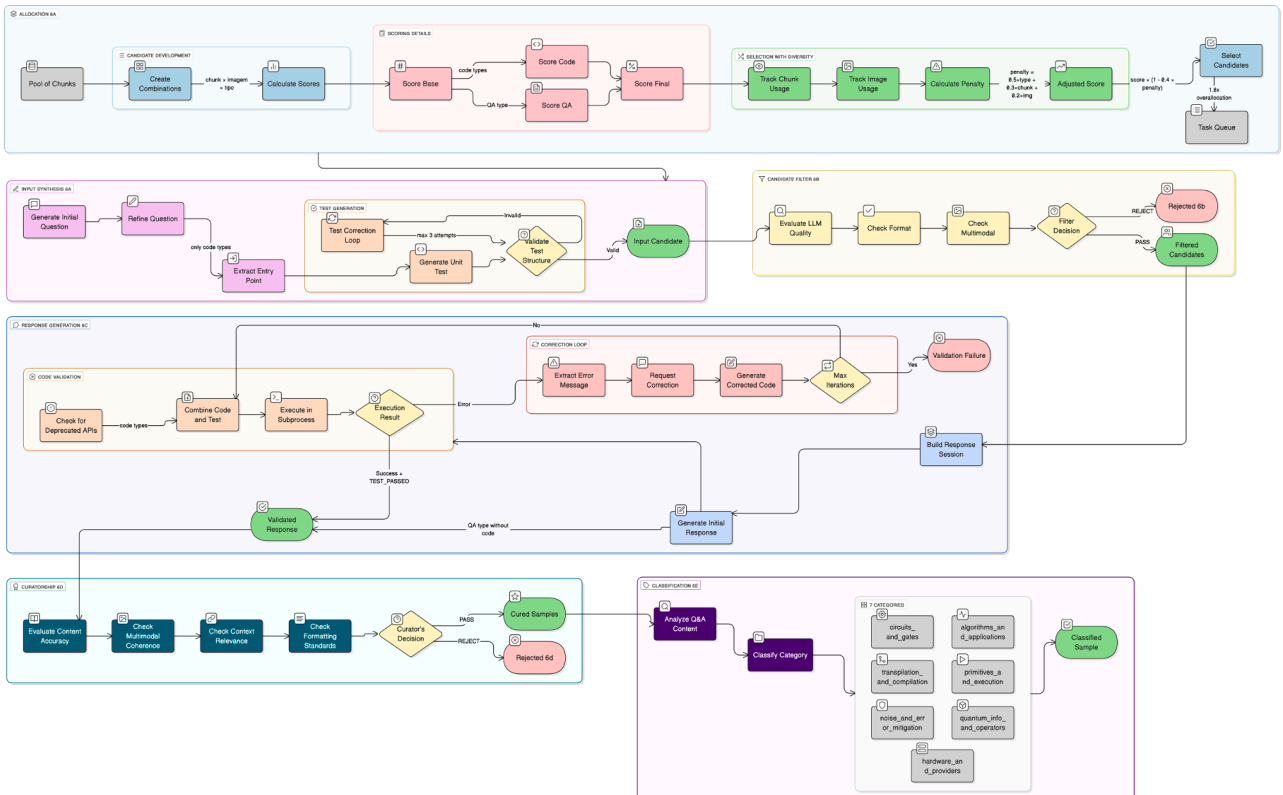


Fig. 2. Detailed sample generation process, including diversity-aware allocation, input synthesis, executable validation, and post-processing.

Table 1

Main data sources used in dataset construction.

Source	Description	License
Qiskit Documentation	Official documentation (guides and tutorials)	Apache 2.0
Qiskit Community	Educational repositories and challenges	Apache 2.0
Qiskit Machine Learning	Quantum machine learning tutorials	Apache 2.0
Qiskit Tutorials	Official exercises and solutions	Apache 2.0
QDC Challenges 2025	Qiskit Developer Challenge problems	Apache 2.0
QGSS 2025	Quantum Global Summer School labs	Apache 2.0
Mastering Qiskit v2.0	Community educational tutorials	Apache 2.0
Learn Quantum Computing	Public educational notebooks	Apache 2.0
Scientific Papers	Papers on Qiskit and quantum computing	CC BY 4.0
Educational Books	Book chapters with open license	CC BY 4.0

Jupyter notebooks (.ipynb): The cell structure is preserved, maintaining the association between code and execution outputs. Images generated by cells (circuit visualizations, histograms) are extracted and linked to the code that produced them through structured metadata, enabling traceability during subsequent processing.

MDX files (.mdx): The parser interprets JSX components specific to Qiskit documentation, including `<Image>` tags with relative paths. YAML frontmatter is processed for metadata extraction (title, category).

PDF documents: Processed via PyMuPDF with block-level extraction, preserving relative positioning between text and images. The parser automatically detects document type (slides, paper, wiki) and applies specific strategies: for slides, it uses page markers as delimiters; for papers, it respects section hierarchy based on font size. PDFs from scientific papers and course materials (such as QGSS 2025) contribute primarily conceptual diagrams and mathematical equations.

At the end of this stage, image references are replaced with structured markers in the format `[IMAGE:id]`, decoupling textual content from image files and enabling independent processing in subsequent stages.

3.1.3. Visual transcription via VLM

Identified images are processed by a VLM for generation of structured textual descriptions. The specialized prompt instructs the model to describe specific technical elements: quantum circuit topology (qubits, gates, connections), state representations (Bloch spheres, fidelity visualizations), measurement histograms, and mathematical equations. Transcriptions are stored as metadata associated with image identifiers. In parallel, an automatic classifier categorizes each image into eight types: *circuit*, *chart*, *bloch_sphere*, *formula*, *diagram*, *table*, *code*, *output*, or *decorative*.

3.1.4. Visual quality filtering

An LLM-based classifier evaluates the relevance and informational quality of each image for the quantum computing domain. The acceptance criterion considers: (1) presence of relevant technical content (circuits, state visualizations, experimental results); (2) quality of the generated transcription (completeness and precision); and (3) coherence with surrounding textual context. Images classified as decorative or with insufficient transcriptions are excluded, and corresponding references are removed from textual content.

3.1.5. Semantic chunking

Documents are segmented into semantic units (chunks) respecting configurable limits: maximum length in tokens, maximum number of code blocks, and maximum number of images per chunk. The algorithm preserves semantic cohesion through document-type-specific strategies: for notebooks, it keeps complete cells (code and output) grouped; for MDX, it respects header hierarchy; for PDFs, it uses section markers. Each chunk maintains: (1) accumulated code context from previous cells, essential for import dependencies; (2) structured references to filtered images; and (3) text from adjacent chunks for context enrichment.

3.1.6. Content quality filtering

Chunks undergo textual quality evaluation via LLM, applying criteria of: (1) density of technical information related to quantum computing; (2) presence of executable and relevant code; and (3) absence of generic content (isolated imports, indices, bibliographies). Chunks classified as low quality are discarded before the generation stage, reducing noise in the final dataset.

3.1.7. Sample generation with validation

The pipeline core generates training samples in three formats aligned with quantum code benchmarks:

- **Function completion (*function_completion*):** Prompt containing imports, function signature, and docstring; the model completes only the function body. Aligned with Qiskit HumanEval (Vishwakarma et al., 2024).
- **Code generation (*code_generation*):** Textual task description in natural language; the model generates complete code including imports. Aligned with Qiskit HumanEval Hard.
- **Conceptual question (*qa*):** Question about theory or concepts; explanatory textual response, possibly including code examples.

For code types, unit tests are automatically generated in Qiskit HumanEval format: the LLM is prompted to produce a `check(candidate)` function that exercises the generated sample through three categories, with the prompt instructing the model to prefer the simpler categories when the task admits them:

- **Structural checks:** verify shape and identity of the generated `QuantumCircuit` — `num_qubits`, `num_parameters`, register counts, ordered gate names, and operand indices.
- **Functional checks:** verify quantum behavior via `Statevector.equiv()` for state-vector equivalence, `Operator.equiv()` for unitary equivalence, and sampled-distribution closeness for circuits with measurements.
- **API/runtime checks:** the candidate must import and execute without raising, enforcing current Qiskit ≥ 2.0 usage (for example, `ZFeatureMap` and `ZZFeatureMap` accept `feature_dimension` as their circuit-size argument rather than `num_qubits`).

This organization follows the HumanEval precedent (Chen et al., 2021) and the Qiskit HumanEval protocol (Vishwakarma et al., 2024), and is aligned with prior practice in synthetic-data pipelines for code that combine LLM-generated instructions with automatic validation (Wang et al., 2023; Wei et al., 2024). An executable validation loop runs the test against the generated answer for up to N error-based correction iterations; samples that do not pass after exhausting attempts are discarded. After validation, a curation stage evaluates overall quality: LLM-assessed grounding of the answer in the source chunk, multimodal coherence (when applicable), and compliance with formatting standards. This stage is not a substitute for a domain-expert semantic audit.

Sample reliability is supported by three complementary layers of evidence. (i) *Executable verification*: every code sample passes its own generated unit test at ingestion, and tests are themselves exercised against a reference implementation; samples whose tests reject the reference are discarded, and 16.6% of candidates are filtered out at this stage (Table 5). Question-answering samples have no executable layer and rely on the remaining checks. (ii) *LLM-based curation* of grounding in the source chunk, multimodal coherence, and formatting, with 19.0% of validated candidates subsequently removed. (iii) *Iterative author inspection during pipeline development*, in which prompts, filters, and unit-test templates were refined over successive runs; the authors are not quantum-computing domain specialists, so this inspection drives refinement of the automated layers rather than certifying every sample. Sections 4.6 and 4.9.11 provide representative samples drawn from the published

split, and the full dataset is released under Apache 2.0 on the Hugging-Face Hub so that any downstream user can replicate or extend the quality assessment on the exact artifacts used here. Executable validity and LLM curation therefore do not guarantee semantic correctness of every sample. A complementary external check for the code subset is out-of-distribution transfer on text-only Qiskit tasks: Qiskit HumanEval is human-curated by IBM, independent of our source chunks, and the fine-tuned model reaches 41.72% Pass@1 on it (Section 4.9.1). That transfer makes it unlikely that the training codes are predominantly wrong, but it does not certify per-sample semantic correctness, nor does it audit conceptual QA or visual grounding.

3.1.8. Models used in the pipeline

The pipeline employs different specialized models for each stage, selected according to each task's requirements. For visual processing — image transcription, type classification, and quality filtering — Gemini 2.5 Flash (Gemini Team, 2025) was used, a multimodal model optimized for visual comprehension tasks with low latency. For question generation, unit tests, and answers, GPT-oss-120B (OpenAI, 2025) was employed, an open-source version with 120 billion parameters from the GPT family, which demonstrates strong code generation and structured reasoning capabilities. Final curation and candidate filtering also use Gemini 2.5 Flash, leveraging its efficiency for large-scale binary classification.

3.1.9. Post-generation thematic classification

Valid samples are classified into seven mutually exclusive categories through semantic analysis of content:

1. *circuits_and_gates*: Construction, manipulation, and properties of quantum circuits and individual gates
2. *quantum_info_and_operators*: State vectors, unitary operators, density matrices, and quantum information theory
3. *algorithms_and_applications*: Implementation of quantum algorithms (VQE, QAOA, Grover, Shor) and practical applications
4. *hardware_and_providers*: Connection to backends, hardware properties, and IBM Quantum service management
5. *transpilation_and_compilation*: Circuit optimization, pass managers, and compilation for specific hardware
6. *primitives_and_execution*: Use of Sampler and Estimator for circuit execution and observable calculation
7. *noise_and_error_mitigation*: Noise modeling, mitigation techniques (ZNE, TREX), and error characterization

Classification uses semantic analysis of generated content, not just source chunk metadata, ensuring samples are categorized according to the knowledge effectively required to answer them.

3.1.10. Stratified split construction

The dataset is partitioned into train, validation, and test sets with simultaneous stratification across three dimensions: question type, thematic category, and modality (multimodal vs. text). The proportion used is 70%/15%/15%, ensuring balanced distribution of all categorical combinations in each split.

3.2. Diversity-aware allocation strategy

Generation employs an allocation algorithm that maximizes corpus coverage while maintaining target distributions of types and modalities. The algorithm operates in three phases:

Phase 1: candidate construction. For each combination (chunk, image, question type), a suitability score $s(c, i, t)$ is calculated considering: presence of executable code, textual length, image type, and technical density. Candidates are organized in a structure indexed by type and modality.

Phase 2: target calculation with over-allocation. For each combination (t, m) where t is question type and $m \in \{\text{multimodal, text}\}$, the target is calculated as:

$$T_{t,m} = \lceil \rho_t \cdot \mu_{t,m} \cdot N \cdot \alpha \rceil \quad (8)$$

where ρ_t is the target proportion of type t , $\mu_{t,m}$ is the multimodal ratio for (t, m) , N is the total target dataset size, and $\alpha = 1.7$ is the over-allocation factor to compensate for losses in filtering and validation.

Phase 3: selection with diversity penalization. During selection, each candidate's score is adjusted by diversity penalization:

$$s'(c, i, t) = s(c, i, t) \cdot (1 - \lambda \cdot u_c) \quad (9)$$

where u_c is the number of times chunk c has already been used and $\lambda = 0.5$ is the diversity weight. This penalization promotes uniform corpus coverage, avoiding over-utilization of specific chunks.

3.3. Fine-tuning protocol

Fine-tuning was conducted using the ms-swift framework (Zhao et al., 2025), which offers integrated support for PEFT techniques and multimodal processing. The experimental strategy was organized in two phases: a method-selection comparison of LoRA variants under a fixed configuration, followed by a restricted hyperparameter search on the retained variant.

3.3.1. Data preparation

The dataset was converted to JSONL format with turn-based message structure, compatible with instruction models. Each sample contains three components: system message with quantum computing specialization prompt, user message with the question (including image reference when applicable), and assistant message with the expected response. For samples with images, these are referenced by relative path and loaded during dataloader processing.

3.3.2. Base model selection

The base VLM was selected under the following criteria, chosen to keep training within a single-GPU budget and to match the parameter scale of existing specialized quantum code models:

1. **Open weights and permissive license**, to enable reproducible fine-tuning and redistribution.
2. **Parameter scale ≈ 8 B**, matching the scale of Qiskit/granite-8b-qiskit and Qiskit/granite-3.3-8b-qiskit (Dupuis et al., 2024) and fitting LoRA training on a single GPU.
3. **Native VLM architecture** with a visual encoder, projection module, and decoder LLM trained jointly; this criterion rules out text-only specialized baselines (e.g. Qwen2.5-Coder-1.4B-Qiskit) from the fine-tuning pool and reserves them as evaluation-time references.
4. **Strong reported performance on general-purpose OCR and visual-grounding benchmarks** in the corresponding technical reports, as a proxy for the model's ability to read technical diagrams after in-domain specialization (Bai et al., 2025; Chen et al., 2025; Gemma Team, 2025).
5. **Active open ecosystem**, with integrated support in training (ms-swift (Zhao et al., 2025)) and serving (vLLM Kwon et al., 2023) frameworks, reducing engineering overhead and improving reproducibility.

Under these criteria, three candidates were shortlisted: Qwen3-VL-8B-Instruct (Bai et al., 2025), InternVL3.5-8B-MPO (Chen et al., 2025), and MiniCPM-V-2.6 (Yao et al., 2024). Selection among them was based on preliminary zero-shot evaluation on the Qiskit HumanEval benchmarks (results reported in Section 4); Qwen3-VL-8B-Instruct achieved the highest baseline Pass@1 on both QHE and QHE-Hard and was retained for all subsequent experiments. MiniCPM-V-2.6 is competitive on general OCR and grounding benchmarks but, under zero-shot invocation on Qiskit HumanEval, frequently emits code wrapped in markdown

fences or with escaped LaTeX tokens, which depresses execution-based scores and leaves it unfit as a fine-tuning target for this study. A broader comparison across VLM families is not a goal of this study and is noted as future work in Section 3.4.

3.3.3. Infrastructure and base configuration

Experiments were executed on an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96GB VRAM). The base training configuration includes: effective batch size of 32 (16 per device with 2 gradient accumulation steps), AdamW optimizer with cosine scheduler, 10-step warmup, and bfloat16 precision with Flash Attention. The visual encoder (Vision Transformer) was kept frozen in all experiments, applying adaptation only to the projection module and language model layers.

3.3.4. Experimental design: PEFT technique comparison

The first phase compared five LoRA variants with controlled configuration ($r = 16$, $\alpha = 32$, dropout 0.05, one epoch):

- **LoRA vanilla:** standard scaling α/r
- **rsLoRA** (Kalajdziewski, 2023): scaling $\alpha/\sqrt{r}$ for stabilization at higher ranks
- **DoRA** (Liu et al., 2024): decomposition into magnitude and direction
- **PiSSA** (Meng et al., 2024): initialization via SVD decomposition of original weights
- **OLoRA** (Buyukakyuz, 2024): initialization with orthogonal QR decomposition

Additionally, a configuration with frozen aligner module (rsLoRA frozen) was tested to evaluate the impact of this component on visual-linguistic adaptation.

This first phase is a *method-selection* protocol rather than an absolute ranking of LoRA variants. All variants are held under an identical configuration in order to isolate the effect of the variant itself from the effect of rank, epochs, and regularization, which are investigated on the selected variant in Section 3.3.5. Each variant is typically most effective in a specific regime — DoRA at low rank (Liu et al., 2024), rsLoRA at higher ranks where its scaling factor stabilizes training (Kalajdziewski, 2023) — so a variant-specific hyperparameter sweep would be required to measure each method's peak performance. That joint sweep is out of scope here and is listed as future work in Section 3.4.

3.3.5. Experimental design: hyperparameter optimization

The second phase investigated the impact of three variables on model performance and generalization:

- **Adapter rank:** $r \in \{16, 32, 64, 128\}$, with $\alpha = 2r$
- **Number of epochs:** $E \in \{1, 2, 3\}$
- **Regularization:** weight decay (0.01 to 0.10) and dropout (0.05 to 0.15)

Experiment monitoring was performed via Weights & Biases (Biewald, 2020), with evaluation every 20 steps on the validation set. Monitored metrics include validation loss (Eval Loss) and token accuracy (Eval Accuracy), used to identify overfitting signals and select the final configuration.

3.3.6. Training-data ablation

To isolate the contribution of the multimodal portion of the training set from the rest of the pipeline, a second configuration is trained on the text-only subset of the synthetic dataset (3,204 training and 679 validation samples, the samples without an associated image) under the same protocol as the best configuration from the previous phase (rsLoRA $r = 32$, $\alpha = 64$, dropout 0.15, weight decay 0.1, learning rate 2×10^{-4} with cosine schedule, 10% warmup ratio, two epochs, effective batch size 32). Only the training data composition differs. The training and validation trajectories are reported in Section 4.8.3, and the downstream

evaluation of the resulting model alongside the main fine-tuned configurations is reported in Section 4.9.4; together they support the separation of the multimodal training effect from the specialization effect on the text portion.

3.4. Scope and limitations

The methodological study is deliberately bounded, consistent with the contribution stated in Section 1. The scope is recorded here so that the results can be read under the correct framing:

- **Qiskit-oriented corpus and evaluation.** The dataset is generated primarily from public Qiskit documentation and related educational materials, and code tasks target Qiskit ≥ 2.0 . Conceptual question answering may involve general quantum computing notions present in those sources, but the resource is not a survey of other quantum SDKs (e.g., Cirq, PennyLane, Q#).
- **Text-only external benchmarks versus multimodal evaluation.** Qiskit HumanEval and Qiskit HumanEval Hard measure quantum-code specialization on text prompts. They do not measure visual quantum understanding. Claims about visual capability rest on the synthetic multimodal test split and on the text-only training ablation (Section 3.3.6), not on the external benchmarks.
- **No independent domain-expert audit.** Sample quality rests on executable verification, LLM-based curation, and iterative author inspection. No independent quantum-domain expert audit of the synthetic set was performed; executable validity does not guarantee semantic correctness of every sample.
- **Single base VLM.** Fine-tuning experiments use Qwen3-VL-8B-Instruct as the only base model. Alternative architectures (InternVL 3.5, MiniCPM-V-2_6) are evaluated zero-shot as baselines; a broader comparison across VLM families is left to future work.
- **Method-selection protocol for PEFT variants.** The five LoRA variants are compared under a single controlled configuration ($r = 16$, $\alpha = 32$, one epoch) to support selection. Variant-specific hyperparameter optimization is not performed; absolute peak performance of each variant is therefore not claimed. Rankings among variants apply only under that fixed setting.
- **Hyperparameter search restricted to the selected variant.** Rank, epochs, and regularization are varied only on the variant retained from the first phase.
- **Primary metric Pass@1.** External benchmarks and synthetic code tasks report Pass@1 with greedy decoding. Pass@k with $k > 1$ and analyses of prompt robustness are out of scope; a justification for Pass@1 as the primary metric is provided in Section 3.6.2. Absolute ranking against IBM-specialized baselines is therefore conditional on the per-model invocation convention of Section 3.6.1.
- **Single training-data composition.** The joint sensitivity of downstream performance to the dataset-generation hyperparameter λ is not studied; λ is analyzed at the pipeline level (coverage, reuse, type balance) in Section 4.7, which is the level at which it is directly interpretable.

3.5. Reproducibility and artifacts

The dataset, fine-tuned adapters, and source code are released under Apache 2.0.² The repository is organized into three primary packages: a synthetic data pipeline (`synthetic_data`) for corpus ingestion, visual transcription, sample generation, executable validation, and stratified export; a fine-tuning package (`finetune`) for ms-swift configuration and adapter training; and an evaluation package (`evaluate`) with config-driven runners for Qiskit HumanEval, Qiskit HumanEval Hard,

² URLs provided in the title page for double-blind review.

and the synthetic test split. Generation, training, and evaluation are exposed through command-line entry points and declarative YAML configurations; the evaluation stack is dependency-pinned via a lockfile so that Pass@1 remains stable under a fixed Qiskit runtime. Package documentation in the public release covers installation and end-to-end recipes; this subsection states the schema and endpoint roles needed for audit, without a software-architecture exposition.

Each released dataset sample exposes the fields used throughout this paper: `question`, `answer`, `type` ($\in \{\text{function_completion}, \text{code_generation}, \text{qa}\}$), `category` (one of the seven thematic labels), `image` (null for text-only samples), `test_code` and `entry_point` (null for QA), and `source`. Multimodality is defined by the presence of a non-null image. Splits are train / validation / test in a 70% / 15% / 15% target ratio with simultaneous stratification on type, category, and modality, yielding 5837 / 1239 / 1290 samples in the published release.

Code-sample validation is performed inside the generation pipeline by executing the generated `check(candidate)` unit test against the candidate and against a reference implementation, followed by the LLM curation stage described in Section 3. The validation stage and benchmark runners are exposed as command-line entry points in the `synthetic_data` and `evaluate` packages, respectively. Downstream evaluation selects the benchmark, serving endpoint, decoding settings, and the per-model system-prompt policy of Section 3.6.1; the Qiskit HumanEval and synthetic runners score Pass@1 by sandbox execution of the accompanying unit tests, and ROUGE-L for conceptual answers. The training, serving, and evaluation configurations used for the reported tables are included in the public release so that the numerical results can be regenerated without reconstructing the experimental setup from the prose alone.

3.6. Evaluation framework

Evaluation employs two complementary strategies to characterize performance on code and conceptual comprehension tasks. All models share the same serving stack and decoding settings, but each is invoked under a fixed native per-model system-prompt policy (Section 3.6.1); absolute cross-model ranking is therefore conditional on that convention.

3.6.1. Serving infrastructure and invocation convention

All models are served with vLLM (Kwon et al., 2023) on a single NVIDIA RTX PRO 6000 Blackwell GPU (96 GB memory). The serving configuration uses bfloat16 precision, the Flash Attention backend, chunked prefill, and prefix caching. Code generation is sampled under greedy decoding (temperature 0) for Pass@1. When Pass@k with $k > 1$ is reported, 20 samples per problem are drawn at temperature 0.2 and Pass@k is estimated with the unbiased estimator of Chen et al. (2021).

Each model is invoked under a native intended-use convention rather than a single shared system prompt imposed on every model. Fine-tuned models and the three baseline VLMs receive the concise system prompt used during training, so the baseline-versus-fine-tuned comparison isolates parameters and training data. The two IBM-specialized Qiskit models (Qwen2.5-Coder-14B-Qiskit and granite-3.3-8b-qiskit) are called with no eval-side system message; their released chat templates then supply each model's default system content when none is provided, rather than leaving an empty system turn. Prompt format and system-message conventions are known to affect absolute LLM scores and can confound cross-model comparison when a single fixed prompt is imposed (Sclar et al., 2024). Absolute ranking against specialized baselines is therefore reported under this fixed per-model policy; a full prompt-robustness sweep remains out of scope (Section 3.4).

3.6.2. External benchmark evaluation

Qiskit HumanEval (Vishwakarma et al., 2024) enables direct comparison with previous works, including the Qiskit Code Assistant

(Dupuis et al., 2024). The Pass@k metric, presented in Eq. (7), uses the unbiased estimator where n is the total number of solutions generated per problem and c is the number of correct solutions (that pass all unit tests). The primary reported metric is Pass@1 ($k = 1$, $n = 1$), the most rigorous setting, which evaluates the model's ability to generate correct code on the first attempt without multiple sampling; Pass@k with $k \in \{1, 5, 10\}$ and $n = 20$ samples at temperature 0.2 is also reported for the best configurations in Section 4.9.1.

Pass@1 is adopted as the primary metric for three reasons. First, Pass@1 is the most conservative setting of Pass@k: it samples a single solution per problem and leaves no room for self-consistency over multiple samples, so improvements cannot be attributed to sampling variance. Second, token-level semantic similarity metrics such as BLEU and ROUGE are known to correlate poorly with correctness on executable code; Eghbali and Pradel (2022) show that in programming languages the shared n-gram content between unrelated snippets dominates BLEU-style scores and motivates dedicated code-aware metrics, and execution-based verification remains the reference standard used both in the original HumanEval (Chen et al., 2021) and in Qiskit HumanEval (Vishwakarma et al., 2024). Third, Qiskit HumanEval is **human-curated by IBM**, independent of the present data generation pipeline and its source chunks, so performance on this text-only benchmark is direct evidence that the model transfers to curated Qiskit code tasks beyond the training distribution. It does not, by itself, measure visual understanding.

3.6.3. Synthetic dataset evaluation

For the proposed dataset, evaluation differs by question type. For code tasks (function_completion and code_generation), Pass@1 is used with execution of unit tests included in the dataset. For conceptual questions (qa), ROUGE-L (Lin, 2004) is employed, measuring the longest common subsequence between generated response and reference. The test split is used exclusively for final evaluation, not participating in hyperparameter selection or early stopping.

4. Results and discussion

This section details the quantitative and qualitative characterization of the constructed dataset, analyzing the effectiveness of the synthetic generation pipeline from corpus ingestion to sample validation. The analysis demonstrates how filtering and diversity allocation strategies contributed to the robustness of the final set. The resulting dataset, named *Quantum Assistant Dataset*, is publicly available on HuggingFace Hub³ under the Apache 2.0 license, accompanied by the complete pipeline source code on GitHub⁴.

4.1. Source corpus characterization

The diversity and quality of the final dataset are intrinsically dependent on primary sources. The collected corpus comprises 1165 permissively licensed documents (Apache 2.0 and CC BY 4.0), whose distribution is detailed in Table 2. A predominance of MDX files (71.1%) is observed, originating from official Qiskit documentation, which provides the terminological and conceptual foundation. However, analysis reveals that Jupyter Notebooks (.ipynb), while representing 26.4% of files, account for 69.4% of extracted executable code. This concentration reflects the didactic nature of notebooks, which tend to contain complete and functional examples essential for training code generation models.

In contrast, PDF documents (2.5%), primarily scientific papers and course materials (such as QGSS 2025), contribute disproportionately with 68.5% of images. This dichotomy between code sources (Notebooks) and visual sources (PDFs) validates the mixed multimodal ingestion strategy adopted in the methodology.

³ URL provided in title page for double-blind review.

⁴ URL provided in title page for double-blind review.

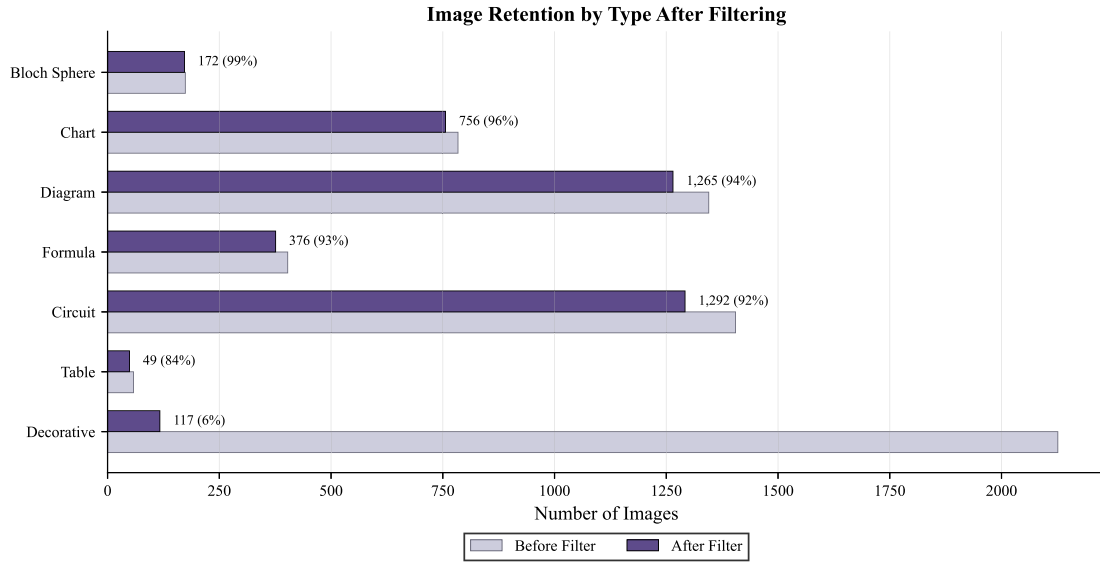


Fig. 3. Image retention rates by type after quality filtering.

Table 2

Source corpus statistics by file type.

Type	Files	Code	Images
.ipynb	308 (26.4%)	3876 (69.4%)	1938 (27.4%)
.mdx	828 (71.1%)	1710 (30.6%)	290 (4.1%)
.pdf	29 (2.5%)	0 (0.0%)	4836 (68.5%)
Total	1165	5586	7064

Table 3

Distribution of image types before and after filtering.

Type	Before	After	Retention
circuit	1405	1292	92.0%
diagram	1345	1265	94.1%
chart	784	756	96.4%
formula	403	376	93.3%
bloch_sphere	174	172	98.9%
table	58	49	84.5%
code_output	18	18	100.0%
unknown	717	146	20.4%
decorative	2126	117	5.5%
Total	7064	4192	59.3%

4.2. Visual processing and noise filtering

The effectiveness of the VLM (Gemini 2.5 Flash) in the transcription stage was high, successfully processing 99.5% of the 7064 extracted images. However, subsequent semantic analysis revealed a significant amount of visual noise in the raw corpus. As demonstrated in Table 3, images classified as *decorative* constituted 30.2% of the total, encompassing elements without pedagogical value, such as logos and visual ornaments.

The LLM-based quality filter acted rigorously, discarding 40.7% of images (2,872 units). Filter selectivity varied according to the technical density of the category: while Bloch spheres had 98.9% retention due to their unambiguous relevance to quantum mechanics, only 5.5% of decorative images were retained (borderline cases containing partial diagrams). The result is a purified visual corpus of 4192 images, maximizing informational density for training. Fig. 3 illustrates retention rates, evidencing preservation of critical categories such as *circuit*, *chart*, and *formula*.

Table 4

Distribution of chunks by source type and content.

Type	Chunks	w/ Code	w/ Image	MM
.ipynb	3199	2045 (63.9%)	1157 (36.2%)	1157
.mdx	828	1710 (30.6%)	290 (4.1%)	290
.pdf	2730	0 (0.0%)	4836 (68.5%)	4836
Total	1165	5586	7064	7064

Table 5

Generation flow efficiency by stage.

Stage	Input	Approved (%)
Planning (candidates)	8281 chunks	13,322 candidates
Candidate filtering	13,322	12,379 (92.9%)
Response generation	12,379	10,327 (83.4%)
Quality curation	10,327	8366 (81.0%)

4.3. Semantic segmentation and textual quality

The segmentation strategy, which prioritized semantic cohesion (keeping code cells and outputs grouped in notebooks), resulted in 8875 chunks with a median length of 2261 characters. Table 4 shows that 63.9% of chunks from notebooks contain executable code, validating their potential for code generation tasks. Textual filtering removed 6.7% of segments, eliminating low-relevance passages (indices, bibliographies), ensuring the model is exposed only to high educational value content.

4.4. Generation pipeline efficiency

The synthetic generation pipeline, operating with an over-allocation factor $\alpha = 1.7$, demonstrated robustness against rigorous validation stages. Table 5 summarizes the flow: of the 13,322 planned candidates, the response generation stage achieved an 83.4% success rate. Notably, the 16.6% failure at this stage reflects the effectiveness of the executable verification mechanism, which prevented inclusion of syntactically incorrect or functionally invalid code (API errors, library hallucinations). Final curation further refined the set, resulting in 8366 high-fidelity samples.

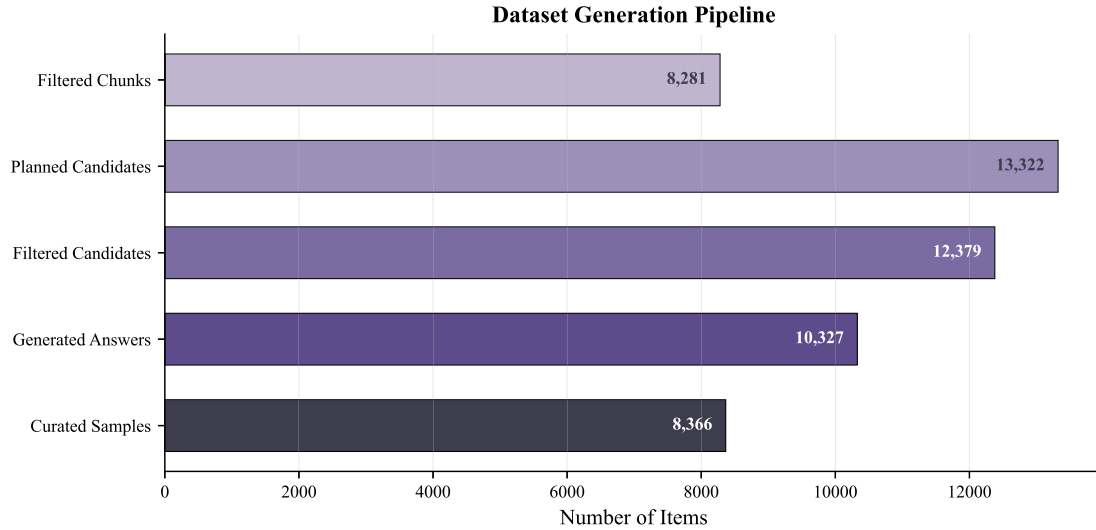


Fig. 4. Synthetic generation pipeline funnel by stage.

Table 6

Consolidated distribution of the final dataset.

Type	Train	Valid.	Test	Total
code_generation	1851	394	408	2653 (31.7%)
function_completion	1758	374	388	2520 (30.1%)
qa	2228	471	494	3193 (38.2%)
Total	5837	1239	1290	8366
Proportion	69.8%	14.8%	15.4%	100%

Table 7

Distribution by thematic category.

Category	Samples	%
circuits_and_gates	2864	34.2%
quantum_info_and_operators	1676	20.0%
algorithms_and_applications	1436	17.2%
hardware_and_providers	815	9.7%
transpilation_and_compilation	641	7.7%
primitives_and_execution	542	6.5%
noise_and_error_mitigation	392	4.7%

Fig. 4 visualizes this processing funnel, highlighting that sample loss throughout the pipeline is indicative of the rigor of quality filters, not process failure.

4.5. Final dataset analysis

The final set presents a planned balance between modalities and task types, as detailed in Table 6. The proportion of samples with images at 45.1% (3,774 samples) represents a significant advance over existing purely textual datasets. Unit test coverage reaches 61.8%, encompassing all code generation and function completion tasks, providing a solid foundation for supervised learning with verification.

The thematic distribution (Table 7) reflects the coverage of the Qiskit documentation and educational materials used as sources, rather than a survey of quantum computing as a whole. The preponderance of the *circuits_and_gates* category (34.2%) is expected, given that circuits are central in those materials. Advanced categories such as *noise_and_error_mitigation* (4.7%) and *transpilation* (7.7%), although less frequent, are represented in sufficient volume to enable specialization in these niches.

Analysis of visual distribution by question type (Fig. 5) reveals an important functional distinction: QA tasks are predominantly accompanied by images (73.0%), while code tasks maintain about 28% visual

support. This aligns with actual practice, where visual interpretation is often the trigger for conceptualization (QA), while pure coding often derives from textual specifications.

4.6. Illustrative dataset examples

To demonstrate the nature and quality of generated samples, we present representative examples of the three task types. Each example illustrates the complete structure of a sample with visual support, including input image, prompt, expected response, and, for code tasks, the automatically generated unit test for validation.

4.6.1. Function completion

Fig. 6 presents a function completion sample from the *circuits_and_gates* category. The prompt provides a stub with imports, function signature, and docstring describing the task; the model must complete only the function body. The image shows a 2-qubit circuit with Hadamard and CNOT gates. The unit test verifies the structure of the returned circuit, gate order, and equivalence of the resulting quantum state.

4.6.2. Code generation

Fig. 7 illustrates a code generation task from the *circuits_and_gates* category. Unlike the previous type, the prompt consists of a textual description in natural language, and the model must generate complete code including imports and function definition. The image shows a Bell state preparation circuit with measurements. The test verifies return types, circuit structure, and equivalence with the $|\Phi^+\rangle$ state.

4.6.3. Question answering

Fig. 8 presents a question answering task from the *quantum_info_and_operators* category. This type requires conceptual interpretation of the image and generation of an explanatory response. The image shows the equation $\text{CNOT}|0+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, and the question asks for explanation of the result and why it produces entanglement. QA samples do not include unit tests, as the response is textual.

Textual samples (without images) follow an identical structure, differing by the absence of visual reference.

4.7. Diversity and coverage

One of the challenges in synthetic generation is avoiding excessive pattern repetition. The allocation algorithm with diversity penalization

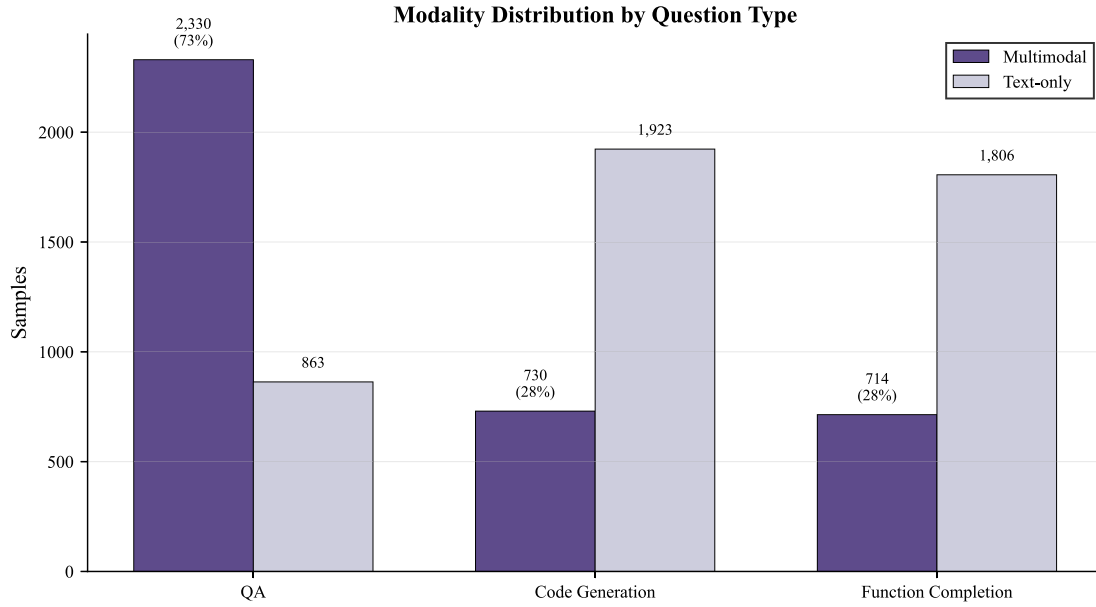


Fig. 5. Modality distribution (text vs multimodal) by question type.

($\lambda = 0.5$) successfully mitigated this risk, achieving 97.4% coverage of unique chunks and 98.5% of available images. The average reuse factor of 1.77 $\times$ indicates that the dataset broadly explores the corpus without saturating specific chunks.

Sensitivity analysis (Fig. 9) confirmed that the value $\lambda = 0.5$ offers the ideal balance. Lower values ($\lambda \rightarrow 0$) would lead to mode collapse, focusing on few “easy” chunks, while higher values ($\lambda \rightarrow 1$) would force use of low-quality chunks just to satisfy diversity metrics.

The λ parameter acts on the generation stage and controls corpus coverage, so its effect is directly interpretable at the pipeline level (coverage, reuse, type balance). Propagating λ to end-task performance would require regenerating the dataset and retraining the model for each value, conflating the data-composition hyperparameter with training-set size and run-to-run variance. The sensitivity analysis is therefore reported at the pipeline level, where λ is directly interpretable; the joint study of data-generation hyperparameters and downstream fine-tuning performance is listed as future work in Section 3.4.

4.8. Fine-tuning experiments

This subsection presents the fine-tuning experiment results, organized in two phases: PEFT technique comparison and hyperparameter optimization.

4.8.1. PEFT method comparison

Fig. 13 synthesizes the results of the five LoRA variants evaluated under the fixed method-selection configuration ($r = 16$, $\alpha = 32$, one epoch). The rsLoRA and DoRA variants achieved the lowest validation loss (0.622), below standard LoRA (0.646) by 3.7% under that setting. Specialized initializations PiSSA (0.657) and OLoRA (0.742) did not demonstrate an advantage under the same controlled configuration.

Although DoRA and rsLoRA presented equivalent performance, computational cost was decisive: DoRA required 2307 seconds versus 1060 seconds for rsLoRA — an overhead of 2.18 $\times$ introduced by the magnitude and direction decomposition.

The convergence curves (Fig. 10) support retaining rsLoRA under this protocol: it converges faster and with greater stability than DoRA at comparable validation loss. Based on this cost-benefit relationship under the fixed configuration, rsLoRA was adopted for the subsequent rank and epoch search.

4.8.2. Rank and epoch optimization

The second phase investigated the impact of adapter dimensionality and training duration.

Rank sensitivity. Fig. 11 presents curves for $r \in \{16, 32, 64\}$. Among the tested ranks, $r = 32$ yields the lowest validation loss: final loss reduced from 0.622 ($r = 16$) to 0.607 ($r = 32$), while $r = 64$ produced a marginal change (0.609). With $r = 32$, the adapter adds 90M parameters (1% of the base model), sufficient to capture Qiskit code patterns and visual correlations under the present budget.

Overfitting analysis. Fig. 12 shows rapid saturation in restricted domain datasets. In the r128-3ep experiment, training loss dropped to 0.058 while validation degraded to 0.789 (+19.5%). Even with r32-2ep, divergence was observed: validation loss rose from 0.607 to 0.638.

Final configuration. The configuration retained for the downstream evaluation is rsLoRA with $r = 32$, two epochs, dropout 0.15 and weight decay 0.1 (r32-2ep). This retention is under the experimental budget of the present study: rsLoRA was selected in the fixed method-selection comparison of five LoRA variants, and rank and epoch were then varied only for that variant (Section 3.3.5; Section 3.4). At $r = 32$, one epoch (r32-1ep) yields a lower validation loss than two epochs (0.607 vs. 0.638, Fig. 12(a)), but the second epoch produces a consistent improvement on the external benchmarks and on the synthetic test split, as reported in Section 4 and analyzed in Section 4.9.8. The two-epoch run is therefore retained as the best configuration under this protocol, while the one-epoch variant is kept in the results tables to document the validation-loss / downstream-metric trade-off. Absolute peak performance of other LoRA variants under variant-specific hyperparameter ranges is not claimed.

4.8.3. Training-data ablation curves

Fig. 14 shows the training and validation curves for the text-only ablation configuration described in Section 3.3.6. Validation loss drops from 0.535 at the first evaluation to 0.436 at the end of the first epoch, then plateaus between 0.436 and 0.449 across the second epoch, ending at 0.437; validation token accuracy stabilizes around 0.879 over the same interval. The reduced sample count (3,204 training samples against 5837 for the full set) therefore does not leave the optimization under-trained; the checkpoint selected by `load_best_model_at_end` sits close to the end of the first epoch, the same zone where the r32-2ep validation curve also stabilizes (Fig. 12(a)), which supports interpreting

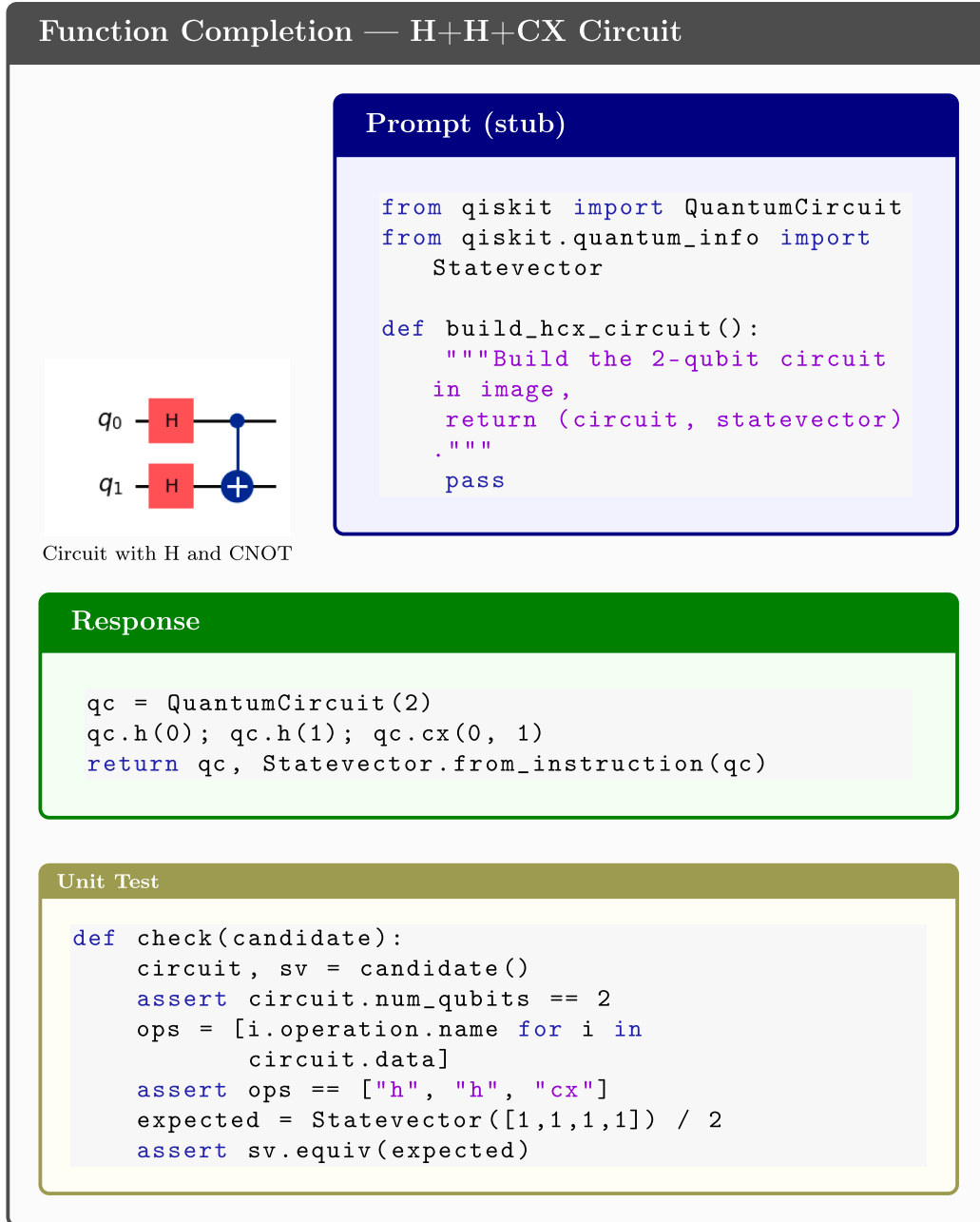


Fig. 6. Function completion example: the model receives a stub with docstring and must implement the function body based on the circuit image.

the downstream comparison in [Section 4.9.4](#) as a controlled measurement of training-data composition rather than of training budget.

4.9. Quantitative evaluation

Quantitative evaluation was conducted on three complementary benchmarks: Qiskit HumanEval (151 function completion problems), Qiskit HumanEval Hard (151 code generation problems), and the synthetic dataset test split (1,290 samples distributed across three task types). The serving stack, sampling strategy, and invocation convention are those defined in [Section 3.6.1](#); Pass@1 ($n = 1, k = 1$) is reported for code tasks and ROUGE-L for conceptual questions.

[Table 8](#) consolidates results across the three benchmarks, with all models evaluated under a single serving configuration (vLLM with bfloat16, Flash Attention, chunked prefill, and prefix caching) on a single RTX PRO 6000 Blackwell GPU and under the per-model invoca-

tion convention defined in [Section 3.6.1](#). The set includes three baseline VLMs (Qwen3-VL-8B-Instruct, InternVL3.5-8B-MPO, MiniCPM-V-2_6), two IBM-specialized Qiskit models (granite-3.3-8b-qiskit at the same 8B parameter scale as our fine-tuning target, Qwen2.5-Coder-14B-Qiskit at 14B), three fine-tuned configurations on Qwen3-VL-8B-Instruct (r32-1ep, r32-2ep, r64-1ep), and one training-data ablation (r32-2ep-textonly) trained on the text-only subset of the synthetic dataset under the same protocol as r32-2ep.

4.9.1. External benchmark performance

On Qiskit HumanEval, the best fine-tuned configuration (r32-2ep) reaches 41.72% Pass@1, above Qwen2.5-Coder-14B-Qiskit at 40.40% and well above granite-3.3-8b-qiskit at 17.88%. The gain over the Qwen3-VL-8B-Instruct base is +17.22 pp. At the same parameter scale (8B plus a 90M adapter against 8B for granite), specialization on the synthetic multimodal dataset produces a +23.84 pp margin over the

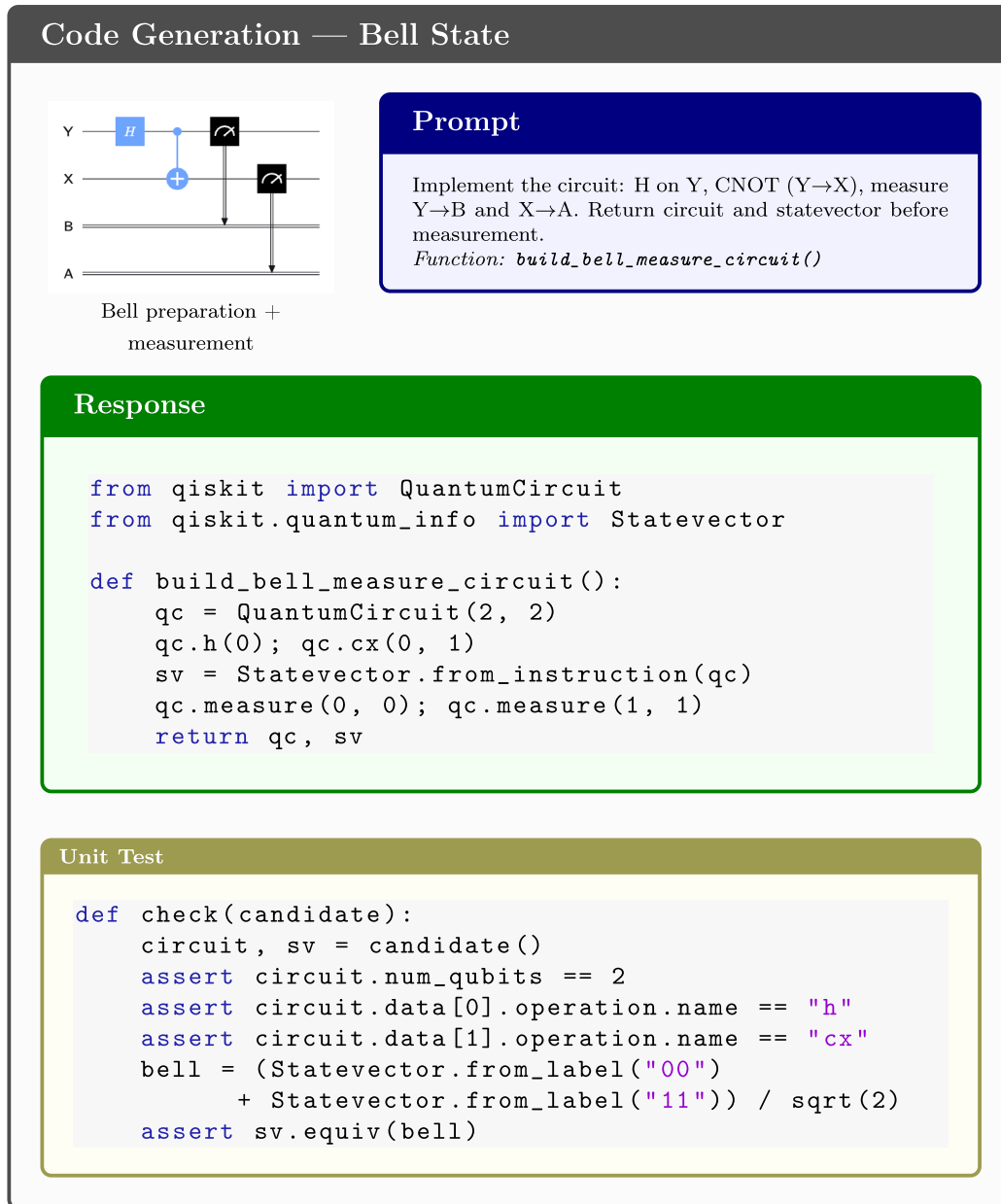


Fig. 7. Code generation example: natural language description with function specification; the model generates complete code.

IBM-specialized 8B model, and at lower parameter count the fine-tuned configuration matches or surpasses the 14B IBM specialized model. The text-only training ablation (r32-2ep-textonly) reaches 41.06% on the same benchmark, within the envelope of the greedy-decoding non-determinism observed in the serving stack.

On Qiskit HumanEval Hard, r32-2ep reaches 30.46% Pass@1, above both Qwen2.5-Coder-14B-Qiskit at 25.83% (+4.63 pp) and granite-3.3-8b-qiskit at 9.93% (+20.53 pp), with a +15.89 pp improvement over the Qwen3-VL-8B-Instruct base.

To complement the greedy Pass@1 metric, Table 9 reports Pass@k with $k \in \{1, 5, 10\}$ for both external benchmarks under 20 samples per problem at temperature 0.2, following the convention of Chen et al. (2021). Pass@k quantifies the probability that at least one of k sampled solutions passes the tests and measures the quality of the solution distribution beyond its single-shot mode.

At $k = 1$, r32-2ep is above both IBM specialized models on both benchmarks (41.95% on QHE against 40.60% for Qwen2.5-Coder-14B-Qiskit and 18.61% for granite-3.3-8b-qiskit; 28.97% on QHE-Hard

against 24.83% and 10.99%). At $k = 10$, the 14B IBM model overtakes (QHE 54.20% vs. 47.59%, QHE-Hard 36.92% vs. 34.22%) while the 8B IBM model does not (QHE 31.52%, QHE-Hard 20.01%): granite's Pass@10 on QHE remains below r32-2ep's greedy Pass@1. The gap between Pass@1 and Pass@10 is about 5.6 pp for r32-2ep, about 13.6 pp for Qwen2.5-Coder-14B-Qiskit, and about 12.9 pp for granite-3.3-8b-qiskit. The fine-tuned configuration concentrates probability mass on its greedy mode; both IBM specialized models distribute probability across more alternatives that still pass the tests, a pattern that is present at both IBM parameter scales (8B and 14B) and is therefore attributable to the continued text-only post-training style rather than to parameter count alone. The text-only training ablation tracks r32-2ep closely across all k , consistent with the two configurations sharing the same adapter mechanism and differing only in training-data composition.

4.9.2. QHE-hard stratified by difficulty

To identify where on Qiskit HumanEval Hard the fine-tuned model differs from the IBM-specialized baselines, problems are stratified by the

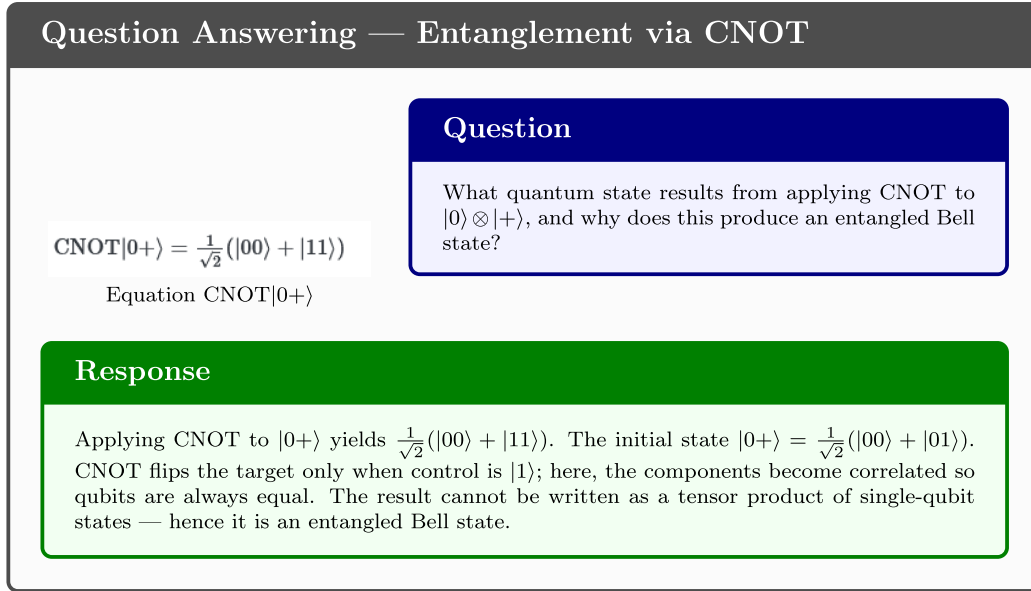


Fig. 8. Question answering example: conceptual interpretation of a quantum equation, explaining the entanglement mechanism.

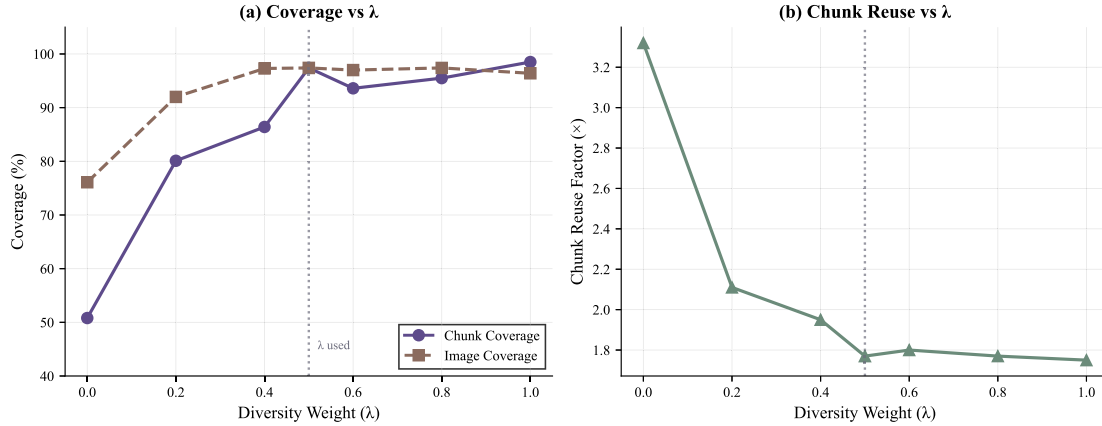


Fig. 9. Sensitivity analysis of diversity weight λ .

difficulty_scale field distributed with the benchmark. The dataset contains 79 basic, 67 intermediate, and 5 difficult tasks (151 total). Table 10 reports greedy Pass@1 per level for the models in Table 8.

Three observations follow. First, r32-2ep overtakes Qwen2.5-Coder-14B-Qiskit on QHE-Hard with the margin concentrated on the basic tier (+7.60 pp: 40.51% vs. 32.91%) and smaller on the intermediate tier (+1.50 pp: 20.90% vs. 19.40%); the difficult tier does not discriminate between the models because no evaluated system solves any of the five difficult tasks. Second, the matched-scale comparison with granite-3.3-8b-qiskit is uniform across tiers (+27.85 pp on basic, +13.44 pp on intermediate), consistent with the headline overall margin. Third, the text-only ablation (r32-2ep-textonly) is -5.07 pp relative to r32-2ep on the basic tier and $+2.98$ pp on the intermediate tier, so the multimodal training signal contributes a measurable lift at the basic tier even though QHE-Hard itself is text-only.

4.9.3. Synthetic dataset performance

On the synthetic test split, fine-tuned configurations are consistently above baselines and IBM-specialized models. For r32-2ep, function completion Pass@1 reaches 57.47% (up from 38.66% for the Qwen3-VL-8B-Instruct base, +18.81 pp) and code generation Pass@1 reaches 45.59% (up from 23.77%, +21.82 pp). On conceptual questions, ROUGE-L moves

from 16.76% for the base to 38.48% (+21.72 pp). Across all 1290 test samples, overall Pass@1 is 51.38%.

Task-type patterns repeat across fine-tuned configurations: function completion remains higher than code generation, reflecting the partial stub in the prompt; code generation benefits most from training as it requires the full program structure. Question answering ROUGE-L is close across the fine-tuned configurations (36.08%–38.48%) and improves uniformly over the Qwen3-VL-8B-Instruct base.

4.9.4. Multimodality impact

Comparison across modality splits of the synthetic test set shows that the gain of fine-tuning is concentrated on samples with an image. For r32-2ep, Pass@1 on multimodal samples is 64.29% against 46.33% on text-only samples ($\Delta = +17.96$ pp). The same gap is smaller for baselines (Qwen3-VL-8B: +5.90 pp; InternVL3.5-8B-MPO: +12.82 pp) and narrows under the training-data ablation (r32-2ep-textonly: +9.26 pp), consistent with the multimodal training signal contributing additional capability on top of what the base VLM already exposes.

To isolate that contribution, the r32-2ep-textonly configuration is trained on the text-only subset of the synthetic dataset (3204 training / 679 validation samples) under the same rsLoRA, optimizer, and schedule used for r32-2ep. The two configurations differ only in training-data composition. Table 11 compares their Pass@1 per modality split. On

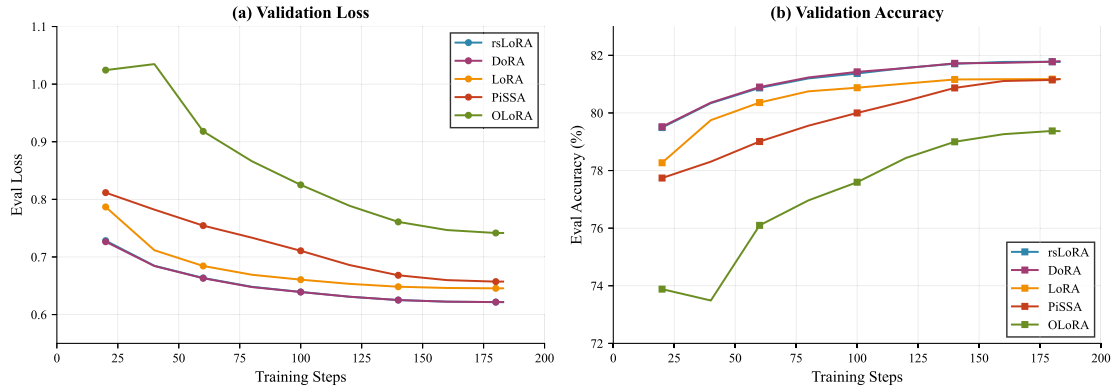


Fig. 10. Validation loss convergence curves for PEFT variants.

Table 8

Consolidated evaluation results across all benchmarks.

Model	QHE	QHE-H	S-FC	S-CG	S-QA	Text	MM
<i>Fine-tuned models</i>							
Qwen3-VL-FT (r32, 2ep)	41.72	30.46	57.47	45.59	38.48	46.33	64.29
Qwen3-VL-FT (r32, 1ep)	38.41	26.49	53.09	42.65	37.89	44.06	57.14
Qwen3-VL-FT (r64, 1ep)	37.75	24.50	51.55	42.16	38.26	41.78	59.38
<i>Training-data ablation</i>							
Qwen3-VL-FT (r32, 2ep, textonly)	41.06	29.14	55.41	47.55	36.08	48.78	58.04
<i>Specialized (text-only) models</i>							
Qwen2.5-Coder-14B-Qiskit ^a	40.40	25.83	46.40	24.49	17.45	35.14	—
granite-3.3-8b-qiskit ^a	17.88	9.93	26.62	15.65	20.18	20.98	—
<i>Baseline VLMs</i>							
Qwen3-VL-8B-Instruct	24.50	14.57	38.66	23.77	16.76	29.37	35.27
InternVL3.5-8B-MPO	18.54	9.93	30.41	14.95	23.81	18.88	31.70
MiniCPM-V-2.6 ^b	1.32	0.66	5.41	4.90	25.15	4.02	8.04

QHE: Qiskit HumanEval (Pass@1); QHE-H: HumanEval Hard (Pass@1); S-FC/S-CG: synthetic function completion/code generation (Pass@1); S-QA: question answering (ROUGE-L); Text/MM: text-only/multimodal samples (Pass@1). All Pass@1 values in %.

^a Evaluated on the textual subset only (55% of the synthetic test split, 709 samples).

^b Code-generation output is frequently wrapped in markdown fences or escaped LaTeX tokens under zero-shot invocation with the shared system prompt, which depresses execution-based scores; the ROUGE-L column on conceptual answers, which tolerates formatting noise, is within 2 pp of the best baseline.



Fig. 11. Impact of adapter rank on validation loss.

text-only samples, removing the multimodal training data produces a +2.45 pp effect in favor of the textonly configuration, attributable to specialization on the remaining subset. On multimodal samples, removing it produces a −6.25 pp drop, attributable to the multimodal portion

Table 9

Pass@k on Qiskit HumanEval (normal and hard) with $n = 20$ samples per problem at temperature 0.2. Pass@k computed with the unbiased estimator of Chen et al. (2021).

Model	QHE			QHE-Hard		
	P@1	P@5	P@10	P@1	P@5	P@10
Qwen3-VL-FT (r32, 2ep)	41.95	47.00	47.59	28.97	33.28	34.22
Qwen3-VL-FT (r32, 2ep, textonly)	40.60	45.88	47.07	26.72	32.15	33.35
Qwen2.5-Coder-14B-Qiskit	40.60	50.77	54.20	24.83	33.68	36.92
granite-3.3-8b-qiskit	18.61	27.65	31.52	10.99	17.53	20.01

Values in %. Best value per column in bold.

of the training set. The two effects roughly cancel on the overall pass rate, which is identical to two decimals (51.38% in both configurations).

The interpretation is that the multimodal training data contributes an image-specific lift of roughly +6 pp on image-bearing problems, while the remaining ≈ 12 pp of the +17.96 pp modality gap observed for r32-2ep reflects the base VLM's existing visual-textual grounding together with the transfer of code-generation patterns learned on the full synthetic set to image-bearing problems.

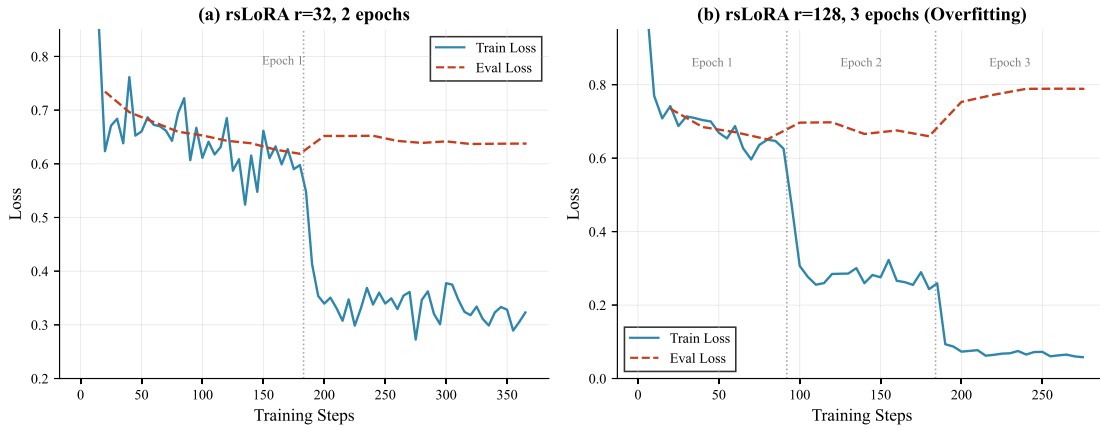


Fig. 12. Overfitting analysis: (a) r32-2ep configuration; (b) r128-3ep configuration.

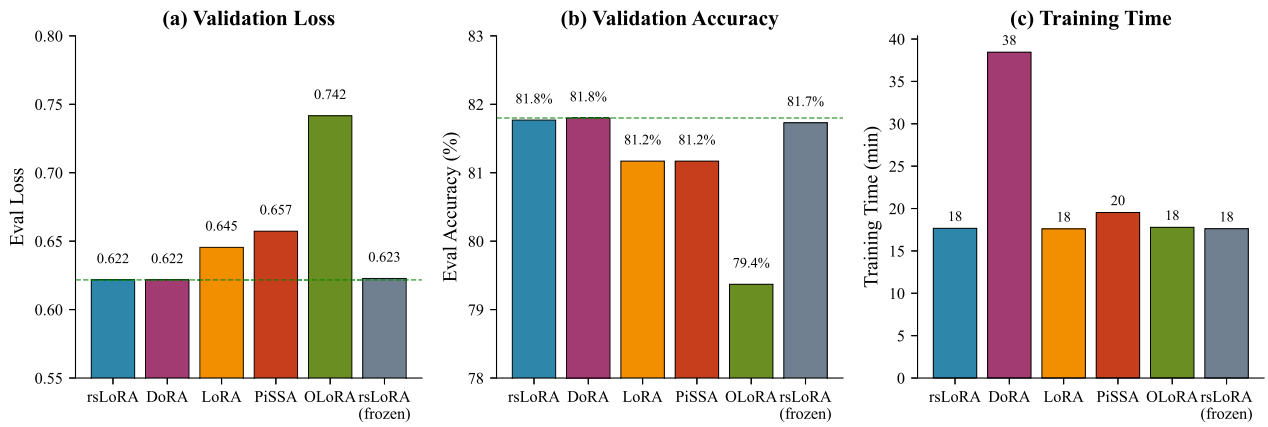


Fig. 13. PEFT variant comparison: (a) final validation loss; (b) token accuracy; (c) training time.

Table 10

Qiskit HumanEval Hard greedy Pass@1 stratified by the benchmark's difficulty_scale field. The *difficult* tier (5 tasks) is scored 0/5 by every model evaluated here and is omitted from the table body (0.00% for all models); values are raw counts over tasks per tier.

Model	Overall (151)	Basic (79)	Intermediate (67)
Qwen3-VL-FT (r32, 2ep)	30.46	40.51	20.90
Qwen3-VL-FT (r32, 2ep, textonly)	29.14	35.44	23.88
Qwen3-VL-FT (r32, 1ep)	26.49	35.44	17.91
Qwen3-VL-FT (r64, 1ep)	24.50	32.91	16.42
Qwen2.5-Coder-14B-Qiskit	25.83	32.91	19.40
granite-3.3-8b-qiskit	9.93	12.66	7.46
Qwen3-VL-8B-Instruct	14.57	22.78	5.97
InternVL3.5-8B-MPO	9.93	13.92	5.97
MiniCPM-V-2_6	0.66	1.27	0.00

Values in %. Best value per column in bold. Counts per tier follow the benchmark's difficulty_scale labels (basic, intermediate, difficult).

4.9.5. Thematic category analysis

Fig. 15 presents Pass@1 stratified by thematic category on the synthetic test split. The fine-tuned configurations stay above baselines across all seven categories, with the largest absolute margins in hardware_and_providers, transpilation_and_compilation, and primitives_and_execution, that is, the topics most exposed to API evolution in Qiskit 2.0 and least covered by the base VLMs' pre-

Table 11

Training-data ablation: same rsLoRA configuration, only the training split differs (full synthetic set vs. text-only subset). Pass@1 on the synthetic test split, broken down by modality.

Evaluation split	r32-2ep (full)	r32-2ep (textonly)	Δ
Text-only samples (709)	46.33%	48.78%	-2.45 pp
Multimodal samples (581)	64.29%	58.04%	+6.25 pp
Overall (1290)	51.38%	51.38%	0.00 pp

Δ is reported as (full - textonly). A positive Δ on the multimodal row indicates that training with image-paired samples improves performance on image-bearing problems; the negative Δ on the text-only row reflects the concentration of the textonly configuration on 3204 text samples instead of the full 5837 training set.

training distribution. Fundamental categories (circuits_and_gates, quantum_info_and_operators) are already addressed in part by the base Qwen3-VL-8B-Instruct and show the smallest relative gains after fine-tuning, while still improving in absolute terms.

4.9.6. Synthesis and comparison with IBM-specialized models

Table 12 contrasts the best fine-tuned configuration, the training-data ablation, the two IBM-specialized Qiskit models, and the Qwen3-VL-8B-Instruct base. r32-2ep surpasses both IBM models across the common benchmarks: on QHE (+1.32 pp vs. Qwen2.5-Coder-14B-Qiskit, +23.84 pp vs. granite-3.3-8b-qiskit), on QHE-Hard (+4.63 pp and +20.53 pp), on synthetic code (+16.09 pp and +30.40 pp), and on synthetic QA ROUGE-L (+21.03 pp and +18.30 pp). The comparison at the matched 8B

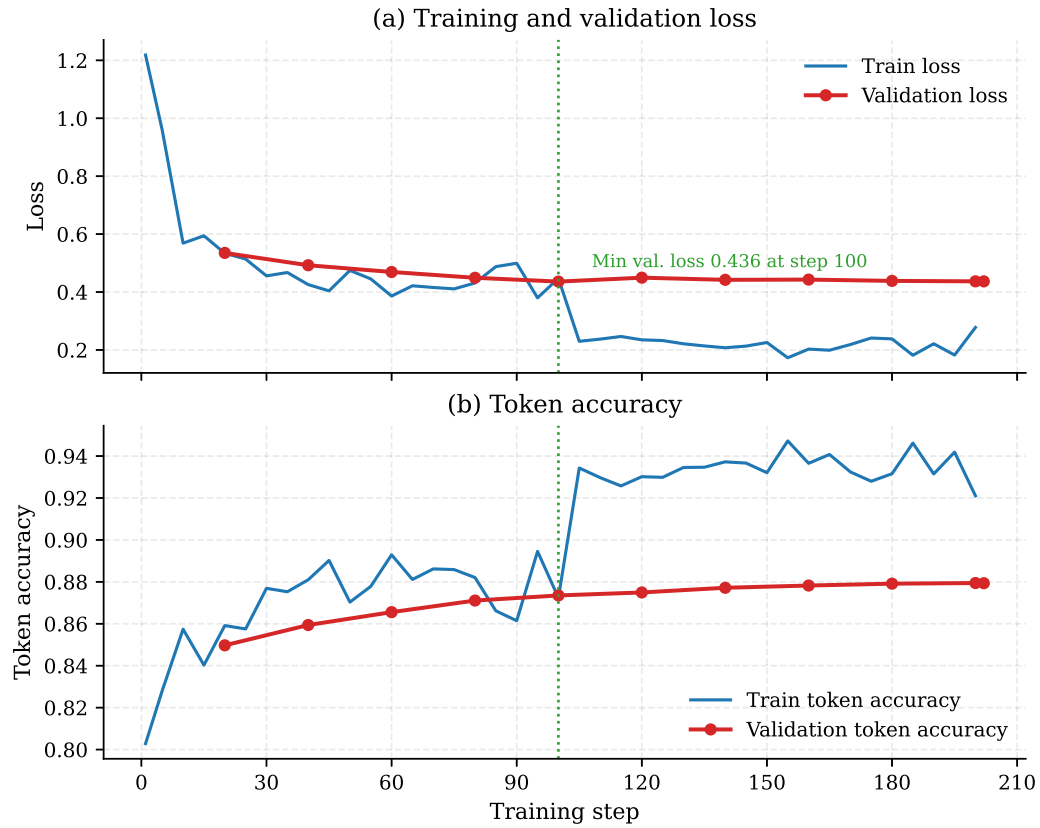


Fig. 14. Training and validation curves for the text-only configuration (r32-2ep-textonly), trained on 3204 text-only samples from the synthetic dataset under the same protocol as r32-2ep. Vertical dotted line marks the checkpoint with the lowest validation loss; the curves plateau over the second half of training.

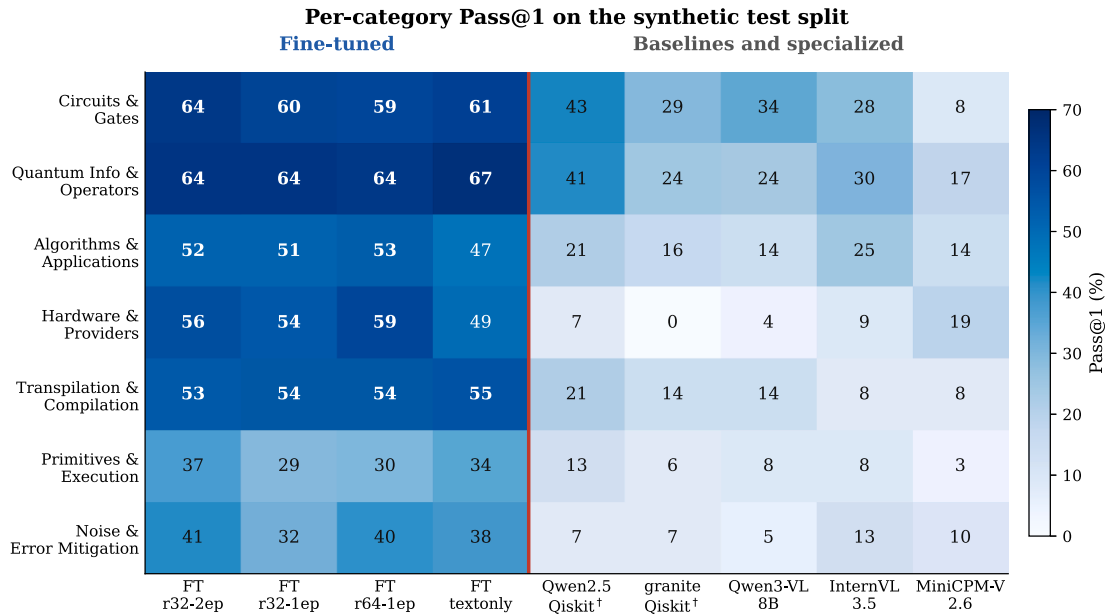


Fig. 15. Per-category Pass@1 on the synthetic test split. Fine-tuned configurations are in the upper block, IBM-specialized models in the middle, and baseline VLMs in the lower block. Cells are unfilled when a category is absent from the split that a given model was evaluated on (e.g., IBM specialized models on the text-only subset).

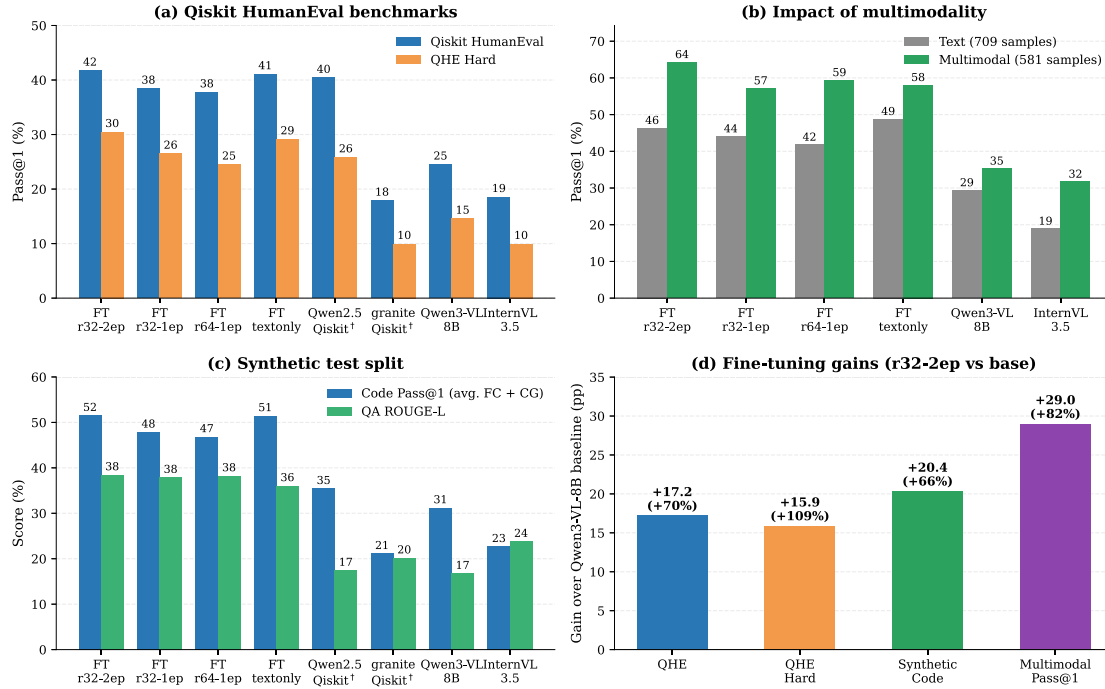


Fig. 16. Visual synthesis of evaluation results: (a) Qiskit HumanEval benchmarks, (b) visual content impact, (c) synthetic dataset, (d) fine-tuning gains over baseline.

Table 12

Head-to-head comparison of the best fine-tuned configuration with the training-data ablation, the two IBM-specialized Qiskit models, and the Qwen3-VL-8B-Instruct base. Best value per column in bold.

Model	Params	QHE	QHE-H	S-Code	S-QA	MM
Qwen3-VL-FT r32-2ep (full)	8B + 90M	41.72	30.46	51.53	38.48	64.29
Qwen3-VL-FT r32-2ep (textonly)	8B + 90M	41.06	29.14	51.48	36.08	58.04
Qwen2.5-Coder-14B-Qiskit	14B	40.40	25.83	35.44 ^a	17.45 ^a	—
Granite-3.3-8B-Qiskit	8B	17.88	9.93	21.13 ^a	20.18 ^a	—
Qwen3-VL-8B-Instruct	8B	24.50	14.57	31.21	16.76	35.27

QHE: Qiskit HumanEval Pass@1; QHE-H: QHE-Hard Pass@1; S-Code: synthetic code Pass@1 (average of function completion and code generation); S-QA: synthetic question answering ROUGE-L; MM: synthetic multimodal subset Pass@1. All values in %.

^a Evaluated on the textual subset only (55% of the synthetic test split).

parameter scale (r32-2ep has 90M adapter parameters on top of the 8B base, granite-3.3-8b-qiskit has full-parameter post-training at 8B) separates the effect of visual specialization on the synthetic dataset from the effect of IBM's textual post-training at the same scale. Fig. 16 visualizes the main results.

4.9.7. Multimodal gain mechanisms

The performance difference between modalities ($\Delta = 17.96$ pp for r32-2ep, Section 4.9.4) indicates that the model learned to exploit visual information specific to the Qiskit-oriented diagrams in the training distribution. Qualitatively, the gain can be attributed to three complementary capabilities: (1) **structural extraction**, by identifying circuit topology directly from diagrams; (2) **symbolic mapping**, by relating state visualizations (Bloch spheres, histograms) with algebraic descriptions; and (3) **parametric inference**, by recovering from figures the numerical values (angles, probabilities, fidelities) that guide correct code generation. The training-data ablation reported in Table 11 localizes +6.25 pp of the modality gap to the multimodal portion of the training set, while the remaining margin reflects the base VLM's pre-existing visual-textual grounding and the transfer of code-generation patterns learned on the text portion of the synthetic set.

4.9.8. Relationship between internal and external metrics

Although the r32-2ep model presents slightly worse validation loss than r32-1ep, it obtains superior results on all three external benchmarks (QHE 41.72 vs. 38.41, QHE-Hard 30.46 vs. 26.49, synthetic overall 51.38 vs. 47.74). Internal metrics based only on average per-token loss therefore do not fully capture the model's ability to produce code that passes unit tests. A second training epoch reinforces useful Qiskit idiomatic patterns, which improves syntactic and structural code correctness even without proportional gains on the validation metric, motivating the r32-2ep selection in Section 3.3.5.

4.9.9. Implications for VLM specialization

The results reveal three central principles for VLM specialization in technical domains. First, **low-rank adapters are sufficient**: with about 1% additional parameters ($r = 32$), it is possible to close much of the gap to a larger model under the present budget. Second, **verified data are more valuable than raw volume**: the use of executable code with unit tests and multi-stage curation favored transfer to curated external code benchmarks. Third, **transfer to independent text-only quantum-code benchmarks is viable**: a model trained on 5837 synthetic samples transferred to 151 curated Qiskit HumanEval problems, suggesting that the dataset captures reusable Qiskit code patterns rather than only

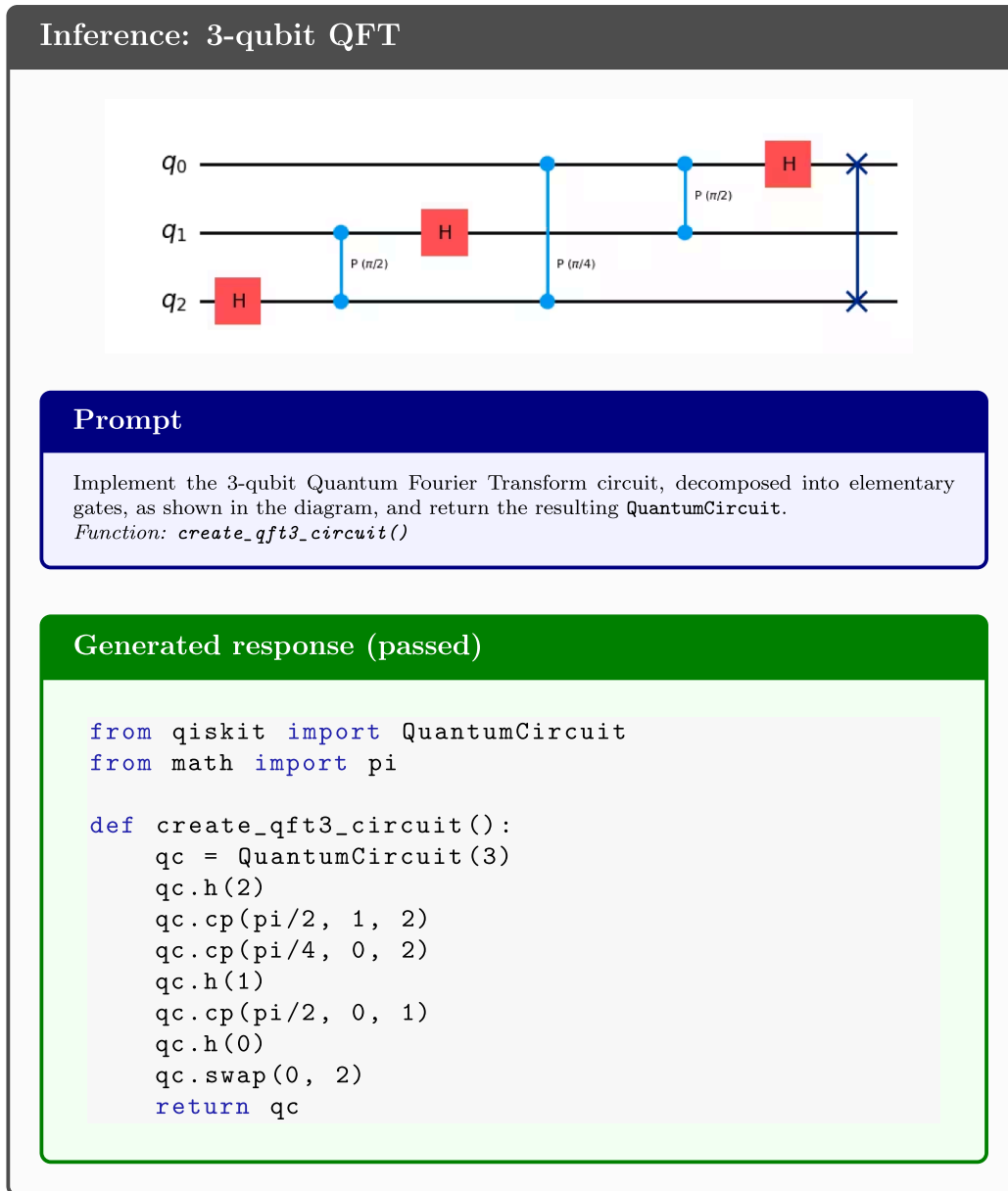


Fig. 17. Inference on code generation: 3-qubit QFT implementation from the diagram decomposed into elementary gates.

pipeline-specific examples. This third point concerns quantum-code specialization on text prompts; it does not by itself establish visual quantum understanding, which is assessed on the synthetic multimodal split (Section 4.9.4).

Because Qiskit HumanEval is human-curated by IBM and is independent of the generation pipeline used here, being neither derived from nor filtered against our source chunks, its Pass@1 scores provide out-of-distribution evidence that the synthetic training signal transfers to curated text-only Qiskit code tasks rather than encoding regularities specific to the training set, and complements the multi-stage curation described in Section 3. No public independent multimodal quantum-code benchmark currently exists against which visual claims could be evaluated externally; those claims therefore rest on the in-distribution multimodal evaluation and the training-data ablation.

The observed asymmetry between code generation and function completion reinforces that visual benefit is greater when the model needs to plan complete code structure from less restrictive descriptions,

while tasks with predefined stubs favor models with greater prior exposure to large textual corpora.

4.9.10. Positioning relative to related work

The proposed dataset differs fundamentally from existing works in purpose and scope. Qiskit HumanEval (Vishwakarma et al., 2024), with 151 function completion problems and its Hard variant with code generation tasks, establishes a manually curated evaluation benchmark for measuring code generation capabilities. This benchmark is used in this work as an evaluation protocol for fine-tuned models, not as a training source. The constructed synthetic dataset serves a complementary purpose: it provides the volume and diversity necessary for supervised fine-tuning, with 5173 code samples verified executably.

Differentiation from the Qiskit Code Assistant (Dupuis et al., 2024) lies in three aspects. First, while IBM's dataset remains proprietary and unpublished, limiting reproducibility, this work makes data, pipeline code, and configurations fully available under the Apache 2.0 license.

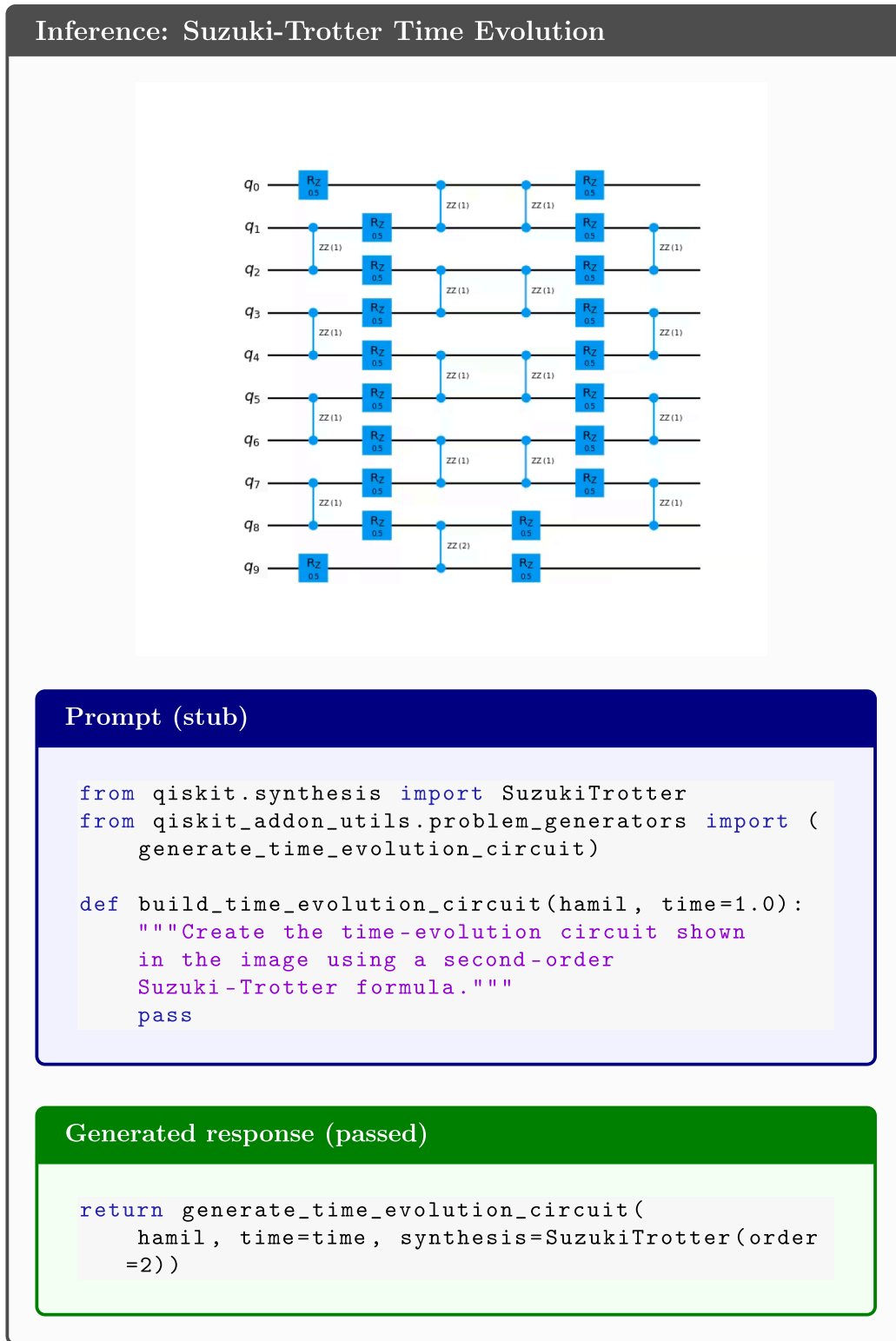


Fig. 18. Inference on function completion: time evolution circuit generation with second-order Suzuki-Trotter formula.

Second, the Qiskit Code Assistant operates exclusively on text, extracting knowledge from GitHub repositories and textual documentation; the proposed dataset integrates 3774 visual-textual samples combining code with circuit diagrams, Bloch spheres, and measurement histograms, enabling image-text-to-text tasks that prior Qiskit code assistants did not address. Third, the synthetic generation methodology with automated executable validation (error-based correction loops and

multi-stage curation) is a methodological contribution for constructing domain-specific code datasets, reproducible for other specialized technical areas.

The quantitative expansion is also significant: compared to the 151 Qiskit HumanEval problems, the proposed dataset offers 2520 function completion samples and 2653 code generation samples, all with unit tests. Additionally, the 3193 question answering samples introduce

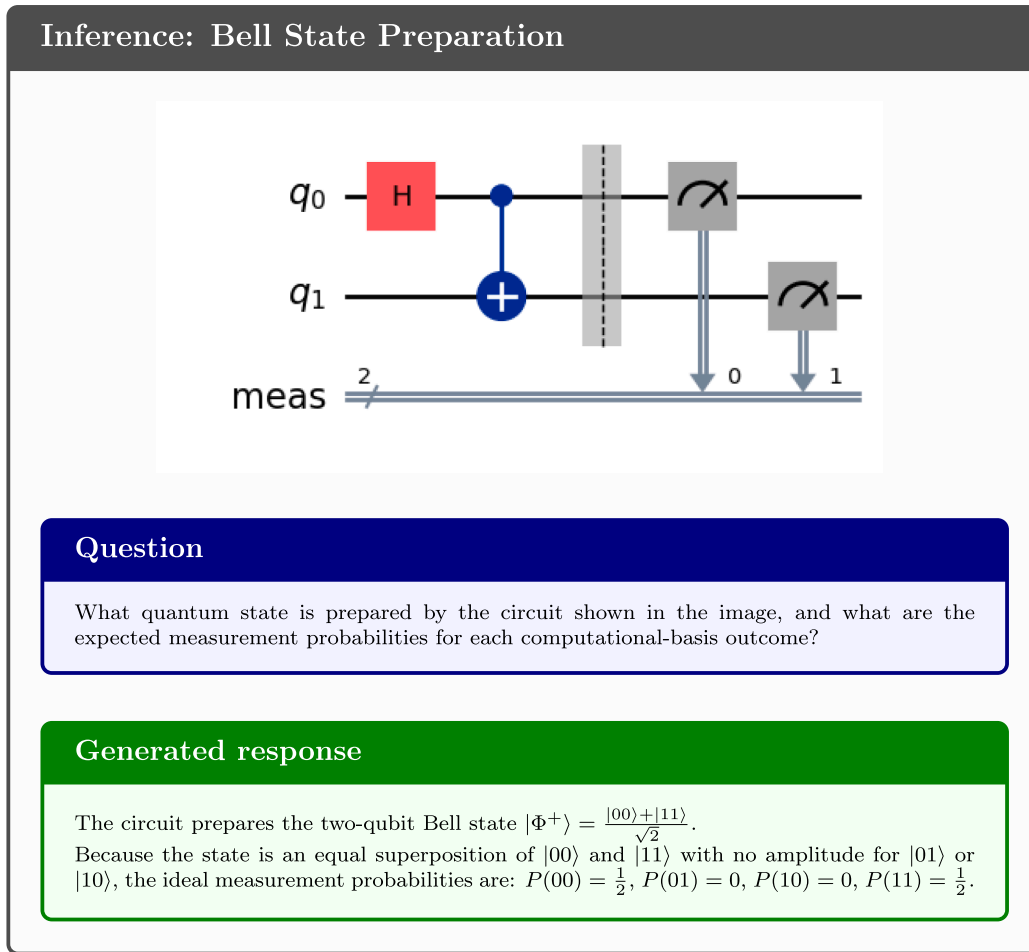


Fig. 19. Inference on question answering: explanation of Bell state preparation and measurement probabilities.

conceptual comprehension capability, enabling training of models for explanation and interpretation, not just code synthesis.

4.9.11. Inference examples

To demonstrate the fine-tuned model's behavior in practical scenarios, we present three inference examples covering each task type. Samples are extracted from the synthetic dataset. Code examples passed their respective unit tests.

Code generation: quantum fourier transform.

Fig. 17 presents a code generation task for the 3-qubit Quantum Fourier Transform (QFT), a fundamental algorithm in quantum computing. The model receives the circuit image decomposed into elementary gates — Hadamards, controlled-phase gates $P(\pi/2)$ and $P(\pi/4)$, and SWAP — and must generate Qiskit code that exactly reproduces this structure. The generated response correctly implements the entire gate sequence.

Function completion: time evolution circuit.

Fig. 18 illustrates a function completion task for generating a time evolution circuit. The model receives the stub with docstring and an image of a 10-qubit circuit extracted directly from the official `qiskit-addon-mpf` API documentation. The circuit shows the structure of a second-order Suzuki-Trotter formula with $R_Z(0.5)$ rotations and $ZZ(1)$ interactions between neighbors. The generated response is identical to the reference, demonstrating correct use of the `qiskit-addon_utils` library.

Question answering: bell state circuit.

Fig. 19 presents a conceptual question about quantum state preparation. The model receives an image showing a circuit with Hadamard

and CNOT gates followed by measurements and must explain the prepared quantum state and its measurement probabilities. The generated response correctly identifies the Bell state and describes the expected outcomes.

5. Conclusion

This work addressed a gap in quantum code assistants: the inability to process visual representations — circuits, Bloch spheres, and histograms — that permeate Qiskit documentation and related educational materials. The contributions are, in order of emphasis: (1) the first public multimodal dataset for Qiskit-oriented quantum programming, with 8366 samples across seven thematic categories, 45% with visual support; (2) a synthetic generation pipeline with executable validation and multi-stage curation, designed for reuse in other specialized technical domains; and (3) a feasibility study of VLM specialization via low-rank LoRA adapters under a controlled method-selection protocol, demonstrating that approximately 1% additional parameters is sufficient for effective domain knowledge transfer on the tasks evaluated here.

The experiments identified rsLoRA with $r = 32$ and two epochs as the best configuration under this experimental budget and method-selection protocol. On the text-only external benchmarks, gains over the Qwen3-VL-8B-Instruct base are +17.22 pp on Qiskit HumanEval and +15.89 pp on QHE-Hard. On the synthetic test split, gains are +20.35 pp overall and +17.96 pp on multimodal samples versus text-only ones. A text-only training ablation under the same rsLoRA protocol attributes +6.25 pp of that modality gap to the multimodal portion of the training set, separating it from the base VLM's pre-existing visual-textual grounding.

Compared to the IBM-specialized Qiskit models, the fine-tuned configuration surpasses both Qwen2.5-Coder-14B-Qiskit and granite-3.3-8b-qiskit across the common benchmarks (QHE, QHE-Hard, synthetic code, synthetic QA) at a matched 8B parameter scale for the adapter base under the per-model invocation convention of Section 3.6.1, and adds multimodal capability that neither IBM model offers. Category analysis shows the effect of specialization is most evident on Qiskit 2.0 APIs such as `hardware_and_providers` and `transpilation_and_compilation`, where the absolute gap over the baseline VLMs is largest.

Limitations and future directions

Four limitations merit attention. First, the training volume (5,837 samples) is inferior to industry proprietary corpora, and some categories remain underrepresented (e.g., *noise_and_error_mitigation*, *primitives_and_execution*), bounding absolute Pass@1 in those topics. Second, saturation after one to two epochs bounds the benefit of prolonged training; augmentation techniques (mixup, paraphrase) could mitigate this effect. Third, as stated in Section 3.4, the study uses a single base VLM, a controlled PEFT comparison rather than variant-specific optimization, and Pass@1 as the primary metric; absolute ranking against specialized baselines is also conditional on prompt invocation conventions. Fourth, no independent quantum-domain expert audit of the synthetic dataset was performed; executable verification and LLM-based curation do not guarantee semantic correctness of every sample, and a structured expert audit remains future work.

Direct extensions include: (i) joint sensitivity analysis of the data-generation weight λ and downstream performance, and variant-specific PEFT hyperparameter sweeps; (ii) a controlled comparison across additional base VLMs (InternVL 3.5 at higher capacity, Gemma 3, and others); (iii) image-type stratified analysis once a released dataset schema exposes the image-type label; (iv) extension of the Pass@k protocol to the synthetic split, following the same sampling budget as the external benchmark; (v) a small expert audit of stratified synthetic samples. Broader directions include: corpus expansion with verified repositories and complementary libraries (Qiskit-Aer, Qiskit-Nature) and, where licenses permit, other quantum frameworks; hybrid architectures combining continued post-training with visual adaptation via PEFT; evaluation on debugging and error interpretation tasks; and domain-specific visual-textual alignment via reinforcement learning, exploring techniques such as DPO (Rafailov et al., 2023) and GRPO (Shao et al., 2024).

The dataset, models, and source code are available on HuggingFace⁵ and GitHub⁶ under the Apache 2.0 license, facilitating reproducibility and extension by the community.

CRedit authorship contribution statement

Samuel Lima Braz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization; **João Paulo Reus Rodrigues Leite:** Supervision, Resources, Writing – review & editing, Project administration.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Gemini 3 in order to improve readability and refine certain sections of the manuscript. Additionally, as documented in Section 3 (Methodology), Gemini 2.5 Flash and GPT-oss-120B were used as integral components of the synthetic data generation pipeline for visual transcription of quantum circuit diagrams, generation of question-answer pairs, and creation of unit tests for code validation. All generated content was validated through

automated executable verification and multi-stage curation. The AI models did not generate or interpret any scientific results; all analyses, conclusions, and interpretations were produced and verified by the authors. After using these tools/services, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The Article Processing Charge (APC) for the publication of this research was funded by the *Coordination for the Improvement of Higher Education Personnel* – CAPES (identifier ROR: 00x0ma614). For open access purposes, the authors have assigned a Creative Commons CC BY license to any accepted version of the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank the Federal University of Itajubá (UNIFEI) for the institutional support provided for this research. We are grateful to the Qiskit community and IBM Quantum for maintaining high-quality open-source documentation and educational resources that served as the foundation for dataset construction. We also acknowledge the HuggingFace team for providing the infrastructure for model and dataset hosting, and the broader open-source machine learning community — including the developers of ms-swift, vLLM, and PyTorch — whose tools made this research possible.

Data availability

The dataset, trained models, and source code are publicly available:

- **Dataset:** <https://huggingface.co/datasets/samuellimabraz/quantum-assistant>
- **Models:** <https://huggingface.co/collections/samuellimabraz/quantum-assistant>
- **Source Code:** <https://github.com/samuellimabraz/quantum-assistant>

All resources are released under the Apache 2.0 license.

References

- Aghajanyan, A., Gupta, S., & Zettlemoyer, L. (2021). Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)* (pp. 7319–7328).
- Bai, J., Bai, S., Yang, S., Wang, S., Tan, S., Wang, P., Lin, J., Zhou, C., & Zhou, J. (2025). Qwen3-VL technical report. <https://doi.org/10.48550/arXiv.2511.21631>
- Biewald, L. (2020). Experiment tracking with weights and biases. <https://www.wandb.com/>. Software, version 0.10.0. Accessed: 2025-11-01.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., & Amodei, D. (2020). Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems* (pp. 1877–1901). Curran Associates, Inc. (vol. 33). <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Buyukakyuz, N. (2024). OLoRA: Orthonormal low-rank adaptation of large language models. <https://doi.org/10.48550/arXiv.2406.01775>
- Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., McClean, J. R., Mitarai, K., Yuan, X., Cincio, L., & Coles, P. J. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9), 625–644. <https://doi.org/10.1038/s42254-021-00348-9>

⁵ URLs provided in title page for double-blind review.

⁶ URL provided in title page for double-blind review.

- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G. et al. (2021). Evaluating large language models trained on code. <https://doi.org/10.48550/arXiv.2107.03374>
- Chen, Z., Wang, W., Tian, H., Ye, S., Gao, Z., Cui, E., Tong, W., Hu, K., Luo, J., Ma, Z. et al. (2025). InternVL 3.5: Advanced multimodal foundation models. <https://doi.org/10.48550/arXiv.2508.18265>
- Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W. et al. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3), 220–235. <https://doi.org/10.1038/s42256-023-00626-4>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houshy, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International conference on learning representations*. <https://openreview.net/forum?id=YicbFdNTTy>.
- Duan, H., Long, Y., Wang, S., Zhang, H., Willcocks, C. G., & Shao, L. (2023). Dynamic unary convolution in transformers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11), 12747–12759. <https://doi.org/10.1109/TPAMI.2022.3233482>
- Duan, H., Shao, S., Zhai, B., Shah, T., Han, J., & Ranjan, R. (2025). Parameter efficient fine-tuning for multi-modal generative vision models with Möbius-inspired transformation. *International Journal of Computer Vision*, 133(7), 4590–4603. <https://doi.org/10.1007/s11263-025-02398-3>
- Dupuis, N., Buratti, L., Vishwakarma, S., Forrat, A. V., Kremer, D., Faro, I., Puri, R., & Cruz-Benito, J. (2024). Qiskit code assistant: Training LLMs for generating quantum computing code. In *2024 IEEE LLM aided design workshop (LAD)* (pp. 1–4). IEEE.
- Eghbali, A., & Pradel, M. (2022). CrystalBLEU: Precisely and efficiently measuring the similarity of code. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering (ASE)*. ACM. <https://doi.org/10.1145/3551349.3556903>
- Gao, P., Han, J., Zhang, R., Lin, Z., Geng, S., Zhou, A., Zhang, W., Lu, P., He, C., Yue, X. et al. (2023). LLaMA-Adapter V2: Parameter-efficient visual instruction model. <https://doi.org/10.48550/arXiv.2304.15010>
- Gemini Team (2025). Gemini 2.5: Pushing the frontier with advanced reasoning, multi-modality, long context, and next generation agentic capabilities. <https://doi.org/10.48550/arXiv.2507.06261>
- Gemma Team (2025). Gemma 3 technical report. <https://doi.org/10.48550/arXiv.2503.19786>
- Herman, D., Googin, C., Liu, X., Sun, Y., Galda, A., Safro, I., Pistoia, M., & Alexeev, Y. (2023). Quantum computing for finance. *Nature Reviews Physics*, 5(8), 450–465. <https://doi.org/10.1038/s42254-023-00603-1>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International conference on learning representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- Kain, S., Ranalli, A., Shekhawat, H., Vishwakarma, S., Ali, H., Bharti, K., & Pistoia, M. (2025). Quantum verifiable rewards for post-training qiskit code assistant. <https://doi.org/10.48550/arXiv.2508.20907>
- Kalajdzievski, D. (2023). A rank stabilization scaling factor for fine-tuning with LoRA. <https://doi.org/10.48550/arXiv.2312.03732>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles* (pp. 611–626). ACM. <https://doi.org/10.1145/3600006.3613165>
- Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning (ICML)* (pp. 19730–19742). PMLR. <https://proceedings.mlr.press/v202/li23q.html>
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text summarization branches out: Proceedings of the ACL-04 workshop* (pp. 74–81). Barcelona, Spain: Association for Computational Linguistics. <https://aclanthology.org/W04-1013>
- Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. In *Advances in neural information processing systems* (pp. 34892–34916). Curran Associates, Inc. (vol. 36). https://proceedings.neurips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-decomposed low-rank adaptation. In *Forty-first international conference on machine learning*. <https://openreview.net/forum?id=3d5CIRG1n2>
- Masry, A., Long, D. X., Tan, J. Q., Joty, S., & Hoque, E. (2022). ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the association for computational linguistics: ACL 2022* (pp. 2263–2279). Dublin, Ireland: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.findings-acl.177>
- Meng, F., Wang, Z., & Zhang, M. (2024). PiSSA: Principal singular values and singular vectors adaptation of large language models. In *Advances in neural information processing systems* (pp. 121038–121072). (vol. 37). https://proceedings.neurips.cc/paper_files/paper/2024/hash/db36f4d603cc9e3a2a5e10b93e6428f2-Abstract-Conference.html
- Murillo, J. M., García-Alonso, J., Moguel, E., Barzen, J., Leymann, F., Mauere, W., Piattini, M., Serrano, M. A., & Weder, B. (2024). Challenges of quantum software engineering for the next decade: The road ahead. <https://doi.org/10.48550/arXiv.2404.06825>
- Nielsen, M. A., & Chuang, I. L. (2010). Quantum computation and quantum information: 10th anniversary edition. Cambridge University Press. <https://doi.org/10.1017/CBO9780511976667>
- OpenAI (2025). gpt-oss-120b & gpt-oss-20b model card. <https://doi.org/10.48550/arXiv.2508.10925>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. In *Advances in neural information processing systems* (pp. 27730–27744). Curran Associates, Inc. (vol. 35). https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
- Qiskit contributors (2024a). Qiskit: An open-source framework for quantum computing. <https://github.com/Qiskit/qiskit>. Accessed: 2025-11-01.
- Qiskit contributors (2024b). Qiskit documentation. <https://github.com/Qiskit/documentation>. Accessed: 2025-11-01.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 53728–53741.
- Sciar, M., Choi, Y., Tsvetkov, Y., & Suhr, A. (2024). Quantifying language models' sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. In *International conference on learning representations (ICLR)*. <https://openreview.net/forum?id=Rlu5LyNjXt>
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. <https://doi.org/10.48550/arXiv.2402.03300>
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F. et al. (2023). LLaMA: Open and efficient foundation language models. <https://doi.org/10.48550/arXiv.2302.13971>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998–6008). Curran Associates, Inc. (vol. 30). <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Vishwakarma, S., Harkins, F., Golecha, S., Bajpe, V. S., Dupuis, N., Buratti, L., Kremer, D., Faro, I., Puri, R., & Cruz-Benito, J. (2024). Qiskit HumanEval: An evaluation benchmark for quantum code generative models. In *2024 IEEE International conference on quantum computing and engineering (QCE)* (pp. 1169–1176). IEEE (vol. 1).
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., & Hajishirzi, H. (2023). Self-Instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 13484–13508). Toronto, Canada: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.acl-long.754>
- Wang, Z., Yu, S., Duan, H., Wang, S., Long, Y., & Shao, L. (2026). Rethinking multi-focus image fusion: An input space optimization view. *IEEE Transactions on Image Processing*, 35, 1321–1336. <https://doi.org/10.1109/TIP.2026.3654370>
- Wang, Z., Zhao, L., Zhang, J., Song, R., Song, H., Meng, J., & Wang, S. (2025). Multi-text guidance is important: Multi-modality image fusion via large generative vision-language model. *International Journal of Computer Vision*, 133(7), 4646–4668. <https://doi.org/10.1007/s11263-025-02409-3>
- Wei, Y., Wang, Z., Liu, J., Ding, Y., & Zhang, L. (2024). Magicoder: Empowering code generation with OSS-Instruct. In *Proceedings of the 41st international conference on machine learning* (pp. 52632–52657). PMLR (vol. 235). Proceedings of Machine Learning Research. <https://proceedings.mlr.press/v235/wei24h.html>
- Yao, Y., Yu, T., Zhang, A., Wang, C., Cui, J., Zhu, H., Cai, T., Li, H., Zhao, W., He, Z. et al. (2024). MiniCPM-V: A GPT-4V level LLM on your phone. [arXiv:2408.01800](https://arxiv.org/abs/2408.01800). <https://arxiv.org/abs/2408.01800>
- Zhai, X., Mustafa, B., Kolesnikov, A., & Beyer, L. (2023). Sigmoid loss for language image pre-training. In *IEEE/CVF international conference on computer vision (ICCV)* (pp. 966–975). <https://doi.org/10.1109/ICCV51070.2023.00095>
- Zhao, Y., Huang, J., Hu, J., Wang, X., Mao, Y., Zhang, D., Jiang, Z., Wu, Z., Ai, B., Wang, A. et al. (2025). SWIFT: A scalable lightweight infrastructure for fine-tuning. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 29733–29735). (vol. 39).